

TABLE OF CONTENTS

Executive Summary	i
List of Tables	iii
List of Figures	v
1 INTRODUCTION	1
1.1 Background	1
1.2 Motivation	1
1.3 Scope of the project	2
2 PROJECT DESCRIPTION AND GOALS	3
2.1 Literature review	3
2.2 Research Gap	3
2.3 Objectives	4
2.4 Problem statement	5
2.5 Project plan	6
3 TECHNICAL SPECIFICATION	9
4 SYSTEM DESIGN	13
4.1 System Architecture	13
4.2 Sequence Diagram	13
4.3 State Transition	14
4.4 Use Case	15
4.5 Class Diagram	15
4.6 Data Flow Diagram	16
4.6.1 Data FLow 0	16
4.6.2 Data FLow 1	17
4.6.3 Data FLow 2	18
5 METHODOLOGY	19
5.1 Sites Selection and Description	19

5.2	Dataset Acquisition and Preparation	20
5.3	Model Implementation	21
5.3.1	LSTM	21
5.3.2	GRU	22
5.3.3	TFT	24
6	PROJECT IMPLEMENTATION	27
7	RESULTS AND DISCUSSION	32
8	CONCLUSION AND FURTHER ENHANCEMENTS	35

EXECUTIVE SUMMARY

Accurate ocean wave forecasting is very important for coastal safety, efficient port operations, supply transportation, and sustainable tourism development James et al. (2018). Many of the existing models that are predicting wave forecasting are physics-based and require expensive computation Hasselmann et al. (1988) Booij et al. (1999). Recent advances in deep learning provide alternative approaches to learn nonlinear temporal patterns in oceanographic data LeCun et al. (2015). In this study, historical ocean wave data is examined with respect to six oceanographic parameters using Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), and Temporal Fusion Transformer (TFT) Hochreiter and Schmidhuber (1997) Cho et al. (2014) Lim et al. (2021). The results show that TFT achieves better performance in multi-target ocean wave prediction.

Traditionally, ocean wave forecasts were built using numerical wave models such as WAM or SWAN which employ sophisticated physical equations to simulate the growth and propagation of waves. However, these simulations require expensive computation and have difficulties representing regional anomalies recorded by sensor locations. Most existing research focuses only on prediction of wave height, while safe maritime activities require a multi-target approach involving wave direction and period as well James et al. (2018). This research is motivated by the need to develop a model that can predict six inter-dependent oceanographic variables simultaneously and evaluate the suitability of recurrent and transformer-based architectures.

The analysis uses a dataset of around 43,454 entries collected from buoy observations at Mooloolaba on Australia's eastern coast. The dataset includes six parameters: Sea Surface Temperature (SST), Wave Direction, Peak Energy Wave Period (T_p), Zero Upcrossing Wave Period (T_z), Maximum Wave Height (H_{max}), and Significant Wave Height (H_s). The data was preprocessed to handle missing values and normalized using Min-Max scaling Han et al. (2012). Three deep learning architectures—LSTM, GRU, and Temporal Fusion Transformer—were implemented for one-step-ahead multi-target forecasting Hochreiter and Schmidhuber (1997) Cho et al. (2014) Lim et al. (2021). Model performance was evaluated using RMSE, MAE, and the coefficient of determi-

nation (R^2).

The experimental evaluation demonstrates notable differences in predictive performance across the models. From Table 1, it can be observed that TFT resulted in better performance than both LSTM and GRU across all six parameters. For Significant Wave Height, TFT achieved an RMSE of 0.0824 m compared to 0.0889 m for LSTM and 0.9093 m for GRU. The largest improvement was observed in SST prediction. The R^2 score of TFT was 0.9797, which was the highest among the evaluated models, indicating superior prediction capability.

Table 1: Model Performance Comparison

Feature	LSTM	GRU	TFT	Best Model
Hs	0.0889	0.9093	0.0824	TFT
Hmax	0.2484	1.3672	0.2420	TFT
Tz	0.3416	1.6312	0.3194	TFT
Tp	1.9561	2.5959	1.9391	TFT
Direction	14.3529	18.6922	13.5743	TFT
SST	0.2117	2.0568	0.1148	TFT
Overall	5.9170	7.8073	5.6006	TFT

This research provided a comparative analysis of LSTM, GRU, and Temporal Fusion Transformer for multi-parameter ocean wave prediction. The results demonstrate that TFT achieves better performance for modeling oceanographic time-series data. The corresponding figure is shown below.

LIST OF TABLES

1	Model Performance Comparison	ii
5.1	Statistical Summary of the Dataset	21
7.1	Model Comparison with RMSE	32
7.2	Model Comparison with MAE	32
7.3	Model Comparison with MAPE	33

LIST OF FIGURES

2.1	Models Comparison – Gantt Chart	6
4.1	System Architecture	13
4.2	Sequence Diagram	13
4.3	State Transition	14
4.4	Use Case Diagram	15
4.5	Class Diagram	15
4.6	Data FLOW 0	16
4.7	Data FLOW 1	17
4.8	Data FLOW 2	18
5.1	Location of Mooloolaba in Australia map	19
5.2	Visualisation of the features.	20
5.3	Visualization of the Real and Predicted Values with LSTM model	22
5.4	Visualization of the Real and Predicted Values with GRU model	24
5.5	TFT learning curves	26
5.6	Visualization of the Real and Predicted values with TFT	26
6.1	Loading Data	27
6.2	Deleting NaN	27
6.3	LSTM Implementation	27
6.4	LSTM Learning Visualization	28
6.5	Visualization of the Real and Predicted Values with LSTM model	28
6.6	GRU Data Processing	28
6.7	GRU Implementation	29
6.8	Visualization of the Real and Predicted Values with GRU model	29
6.9	TFT Implementation	30
6.10	TFT Learning	30
6.11	Visualization of the Real and Predicted values with TFT	31

7.1	Model Performance Comparison: LSTM vs GRU vs TFT	34
8.1	Conference Acceptance Letter	54

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Oceanographic forecasting is also a crucial service which has the goal of predicting the development of the sea state to assist in maritime navigation, offshore industrial activities (including resource exploration) and coastal protection. Traditionally, these forecasts were built using numerical wave models (i.e., WAM or SWAN) which employ sophisticated physical equations to simulate the growth and propagation of waves. Practically, however, these simulations require expensive computation and have difficulties to represent regional anomalies that can be recorded by some sensor locations Afzal et al. (2023). With the development of high resolution sensor technology, such as ocean buoys used in this project, large time series datasets are available. These buoys measure parameters including Significant Sea Surface Temperature (SST), Wave Height, Wave Periods. The availability of this data has paved the way for Deep Learning approaches, which can learn the non-linear, temporal patterns of the ocean without the need for manual feature engineering or high-cost physical simulations Adytia et al. (2023).

1.2 MOTIVATION

Most existing research is only concerned with the prediction of wave height Afzal et al. (2023) Hao et al. (2023). But for safe maritime activities, there is a need for a multi-target approach involving wave direction and period as well. This research is motivated by the need to develop a model that can predict six inter-dependent variables at the same time. Many existing studies have already demonstrated that deep learning based models produce better results than physics based models Lawal et al. (2024) James et al. (2018). Now, although LSTMs and GRUs are the most common models for time-series data, they tend to have difficulties with long-term dependencies and do not offer any understanding of which variables (such as SST or Direction) have the most influence on the prediction Dai and Liang (2025). The reason for the implementation of the Temporal Fusion Transformer (TFT) is motivated by its ability to offer understandability through attention mechanisms and its state-of-the-art Variable Selection Networks,

which assist in understanding the most important oceanographic variables for the forecasting process. The improvement of the accuracy of Hmax (Maximum Wave Height) and the direction of the peak predictions can directly contribute to the reduction of the risks associated with maritime accidents and the damage of coastal infrastructure.

1.3 SCOPE OF THE PROJECT

The analysis uses a dataset of around 43,454 entries. The dataset has an extensive preprocessing stage to counter missing data from sensors those are deployed to get the ocean data. The result of the models are calculated to predict a vector of six parameters, those are Sea Surface Temperature (SST), Wave Direction (relative to true north), Peak Energy Wave Period (Tp), Zero Upcrossing Wave Period (Tz), Maximum Wave Height (Hmax), Significant Wave Height (Hs). The project includes the essential development of three different deep learning models. Those are LSTM (multi-layered), GRU, and the Temporal Fusion Transformer (here we are using TFT). The examination is based on RMSE, MAE, MAPE, and R^2 values to identify the best-performing model for real-world use.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

Ocean wave prediction has mostly relied on physics-based and numerical models (eg, spectral wave models and hydrodynamic simulations). But these models are based on ocean physics, so these techniques usually need high computational cost, detailed boundary conditions, and besides this these techniques are struggling with real-time prediction. In recent years, machine learning and deep learning based techniques have given much more accuracy in the field of wave forecasting. Classical ML models like linear regression and support vector machines showed some improvement but they also failed to catch the temporal dependencies present in ocean wave data. To overcome these issues, Recurrent Neural Networks (RNNs) models, have been widely used. In RNNs, we have LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) which show great improvements. These models are quite suited for time-series data as they are based on learn long-term dependencies and non-linear relationships. Several studies show that LSTM and GRU beat the traditional methods in terms of predicting parameters such as significant wave height, maximum wave height, and wave period. Recently, Transformer-based architectures, such as TFT (Temporal Fusion Transformer) have shown great results by utilizing attention mechanisms.

2.2 RESEARCH GAP

1. Handling Corrupted Real-World Data

Many studies assume the availability of clean datasets. However, real oceanographic data often contains corrupted values, missing entries, and invalid measurements, making it difficult to build robust predictive models.

2. Insufficient Multi-Feature Prediction

Most existing approaches focus on predicting a single wave parameter. This limits the model's ability to accurately capture the complex interactions among multiple ocean wave features.

3. Improper Sequence Preparation

In several implementations, LSTM or GRU models are trained on time-series data without proper sequence structuring. This results in models receiving isolated data points or inconsistently sized sequences, which hampers their ability to learn temporal patterns effectively.

4. Limited Comparative Analysis Across Architectures

Many studies evaluate either LSTM/GRU models or Transformer-based models independently. The lack of comparative analysis on the same dataset makes it difficult to identify the most effective architecture.

5. Insufficient Evaluation Metrics

Prior research often relies on a single evaluation metric. A more comprehensive evaluation using multiple metrics such as RMSE, MAE, and visual comparisons provides a more reliable assessment of model performance.

6. Poor Interpretability Through Visualization

Many models are treated as black boxes. Incorporating feature plots, correlation heatmaps, and prediction vs. actual graphs improves interpretability and provides deeper insights into model behavior and accuracy.

2.3 OBJECTIVES

1. Identifying the most accurate deep learning models for ocean wave height prediction using time-series data.
2. Contribute to the safety of the coastal and rural shoreline by early identification of hazardous wave conditions that may damage the coastal settlements.
3. Contribute to the port and harbor operations by providing reliable wave forecasts which helps authorities in ship navigation, cargo handling, and to reduce the operational risks.

4. To intensify marine transportation and fishing activities across costal areas by providing crucial information about sudden wave height variations, as a result this will improve the safety at sea.
5. Protect the tourism management by encourage the authorities to plan beach activities, water sports, and tourist safety measures based on predicted wave conditions.
6. Comparing the effectiveness of LSTM, GRU, and Temporal Fusion Transformer (TFT) models to identify which model give is more precise is ocean wave forecasting.

2.4 PROBLEM STATEMENT

Oceanographic forecasts have a key place in ensuring safety in maritime and offshore activities and coastal defense. Conventional numerical models like WAM and SWAN operate under sophisticated physical laws and consume huge computing power, at the same time being unable to accurately simulate regional irregularities detected by sensors' data.

Due to the growing abundance of ocean buoys with high resolution, there appears to be an opportunity to use deep learning algorithms to achieve better prediction efficiency and accuracy. Yet, most solutions available today are focused on wave height prediction without taking into account the joint prediction of several interrelated oceanographic features like wave period and wave direction.

On top of that, frequently used models like LSTM or GRU suffer from inefficiency in detecting long-range dependencies between parameters and insufficient interpretability as regards the impact exerted by input parameters.

Thus, there appears to be an urgent need in an efficient and explainable deep learning approach for multi-target prediction from time-series data, enabling the forecast of several oceanographic parameters at once.

2.5 PROJECT PLAN

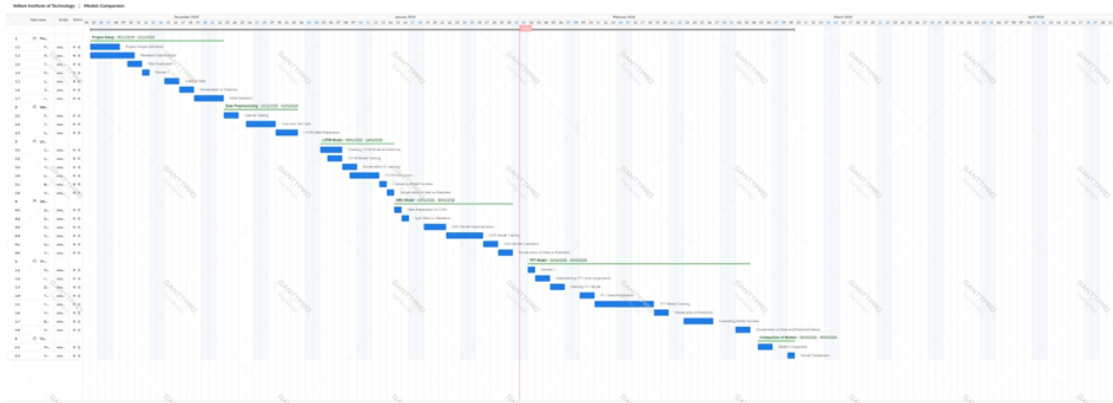


Figure 2.1: Models Comparison – Gantt Chart

PHASE 1: PROJECT SETUP (5 DEC 2025–22 DEC 2025)

During this time we focus on defining the project foundation and we understand the problem domain.

- **Project Scope Definition:** 5 Dec – 8 Dec
- **Research Gap Analysis:** 5 Dec – 10 Dec
- **Title Finalization:** 10 Dec – 11 Dec
- **Review 1:** 12 Dec – 12 Dec
- **Loading Data:** 15 Dec – 16 Dec
- **Visualization of Features:** 17 Dec – 18 Dec
- **Initial Statistics:** 19 Dec – 22 Dec

PHASE 2: DATA PREPROCESSING (23 DEC 2025 – 1 JAN 2026)

During this phase we prepare the dataset for deep learning models.

- **Feature Scaling:** 23 Dec – 24 Dec
- **Train and Test Split:** 26 Dec – 29 Dec
- **LSTM Data Preparation:** 30 Dec – 1 Jan

PHASE 3: LSTM MODEL DEVELOPMENT (5 JAN 2026 – 14 JAN 2026)

This phase involves designing, training, and evaluating the LSTM model.

- **Creating LSTM Model Architecture:** 5 Jan – 6 Jan
- **LSTM Model Training:** 6 Jan – 7 Jan
- **Visualization of Learning:** 8 Jan – 9 Jan
- **LSTM Predictions:** 9 Jan – 12 Jan
- **Evaluating Model Success:** 13 Jan – 13 Jan
- **Visualization of Real vs Predicted Values:** 14 Jan – 14 Jan

PHASE 4: GRU MODEL DEVELOPMENT (15 JAN 2026 – 30 JAN 2026)

We focus on implementing and evaluating the GRU model.

- **Data Preparation for GRU:** 15 Jan – 15 Jan
- **Split Data for Validation:** 16 Jan – 16 Jan
- **GRU Model Implementation:** 19 Jan – 21 Jan
- **GRU Model Training:** 22 Jan – 26 Jan
- **GRU Model Evaluation:** 27 Jan – 28 Jan
- **Visualization of Real vs Predicted Values:** 29 Jan – 30 Jan

PHASE 5: TFT MODEL DEVELOPMENT (2 FEB 2026 – 3 MAR 2026)

We will implement the Temporal Fusion Transformer (TFT) model.

- **Review 2:** 2 Feb – 2 Feb
- **Implementing TFT Core Components:** 3 Feb – 4 Feb
 - Gated Linear Unit
 - Gated Residual Network

- Variable Selection Network
 - Interpretable Multi-Head Attention
- **Defining TFT Model:** 5 Feb – 6 Feb
- **TFT Data Preparation:** 9 Feb – 10 Feb
- **TFT Model Training:** 11 Feb – 18 Feb
- **Visualization & Prediction:** 19 Feb – 20 Feb
- **Evaluating Model Success:** 23 Feb – 26 Feb
- **Visualization of Real and Predicted Values:** 2 Mar – 3 Mar

PHASE 6: COMPARISON OF MODELS (5 MAR 2026 – 9 MAR 2026)

In this phase, We will focus on analyzing and comparing all implemented models.

- **Model Comparison:** 5 Mar – 6 Mar
- **Visual Comparison:** 9 Mar – 9 Mar

CHAPTER 3

TECHNICAL SPECIFICATION

3. ANALYSIS AND REQUIREMENTS

3.1. Analysis

The ocean wave dataset has been used in this study. We have used real-world data as a result they contain corrupted as well as empty values. Ocean wave data is cleaned by removing invalid values and applying normalization to ensure stable model training.

Correlation analysis and visual inspection are utilized to understand the dependencies among wave parameters.

Fixed-length temporal windows are constructed using a sliding window approach to preserve sequential information.

LSTM and GRU models capture sequential dependencies through recurrent memory mechanisms.

The TFT model holds attention mechanisms to focus on dominant time steps and features which improves understandability.

Model predictions are evaluated using error metrics such as RMSE and MAE, along with visual comparison plots.

3.2 Requirements

3.2.1. Functional Requirements

- The system shall be capable to collect and process historical ocean wave data for prediction.
- The system shall clean and preprocess raw data by dealing with corrupted values, invalid data, and missing datas.
- The system shall be capable of scaling and normalize data for deep learning model training. The system shall be able to produce fixed-size temporal sequences using a sliding window technique for time series learning.

- The system shall be able to design and implement various deep learning models, such as LSTM, GRU, and Temporal Fusion Transformer (TFT), for ocean wave forecasting prediction.
- The system shall be able to handle multivariate input data to learn the relationships between various ocean wave parameters.
- The system shall be able to test the performance of models using error measures like RMSE and MAE to gain accuracy.
- The system shall be able to provide visualization of results, such as actual vs predicted wave height and correlation plots to identify the accuracy of the models.
- The system shall enable the comparison of prediction performance of various models.

3.2.2. Non-Functional Requirements

- **Performance Requirements:** The system should be good at wave height predictions in a reasonable time frame which will help us in operational purposes.
- **Accuracy Requirements:** The system should be able to reduce the prediction error and ensure stable predictions for varying time intervals. The predictions should be accurate enough to support decision-making in coastal safety and marine activities.
- **Scalability Requirements:** The system should be able to process large amounts of historical data with precise performance. The system architecture should be scaled to include more wave parameters in the future.
- **Reliability Requirements:** The system should be able to produce consistent results when trained under the same circumstances. The system should be able to process incomplete or corrupted data without crashing.
- **Usability Requirements:** The system should be easy to use with latest Python tools and latest TensorFlow versions.

3.2.3. Software Requirements

- **Programming Language**
 - Python (version 3.8 or above)
- **Data Processing and Analysis Libraries**
 - NumPy (Numerical Computations)
 - Pandas (Data Handling and Preprocessing)
- **Visualization Tools**
 - Matplotlib (Plotting and Result Visualization)
 - Seaborn (Correlation heatmaps and statistical plots)
- **Machine Learning and Deep Learning Frameworks**
 - Scikit-learn (Scaling, Matrices, and Data Splitting)
 - TensorFlow (Implementation of LSTM, GRU, TFT models)

3.2.4. Hardware Requirements

- **Processor**
 - Multi-core CPU (Intel i5 / AMD Ryzen 5 or higher recommended)
- **Memory**
 - Minimum 8 GB RAM
 - Recommended: 16GB for smoother training of deep models
- **GPU**
 - NVIDIA GPU with CUDA support for faster model training
- **STORAGE**
 - At least 20-30 GB free disk space for dataset, models, and logs

3.2.5. Data Requirements

Historical ocean wave data including multiple wave-related parameters such as wave height and wave period.

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

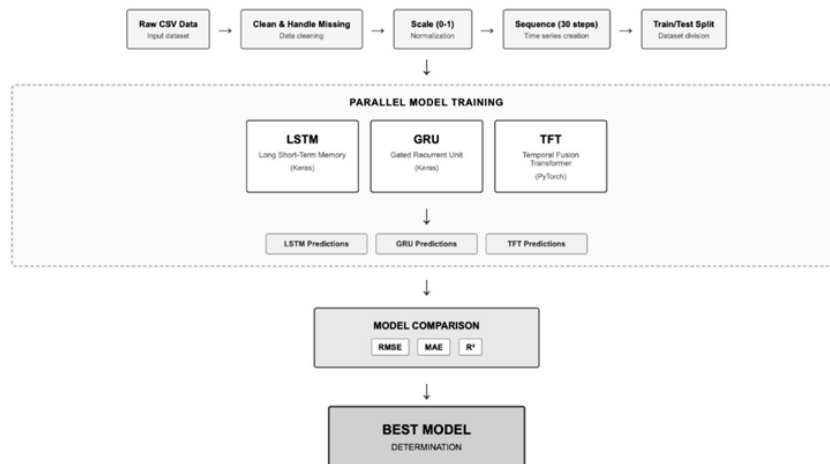


Figure 4.1: System Architecture

4.2 SEQUENCE DIAGRAM

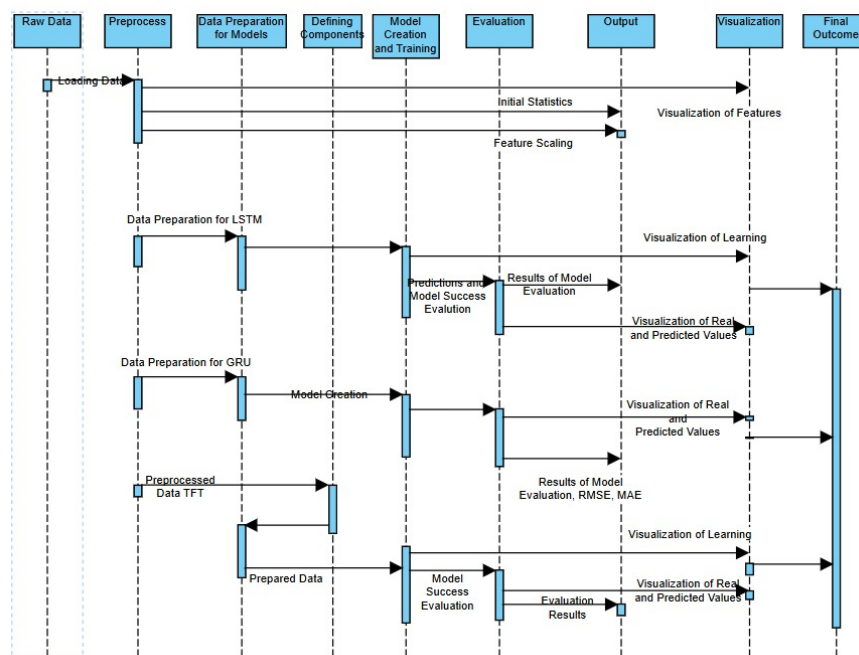


Figure 4.2: Sequence Diagram

4.3 STATE TRANSITION

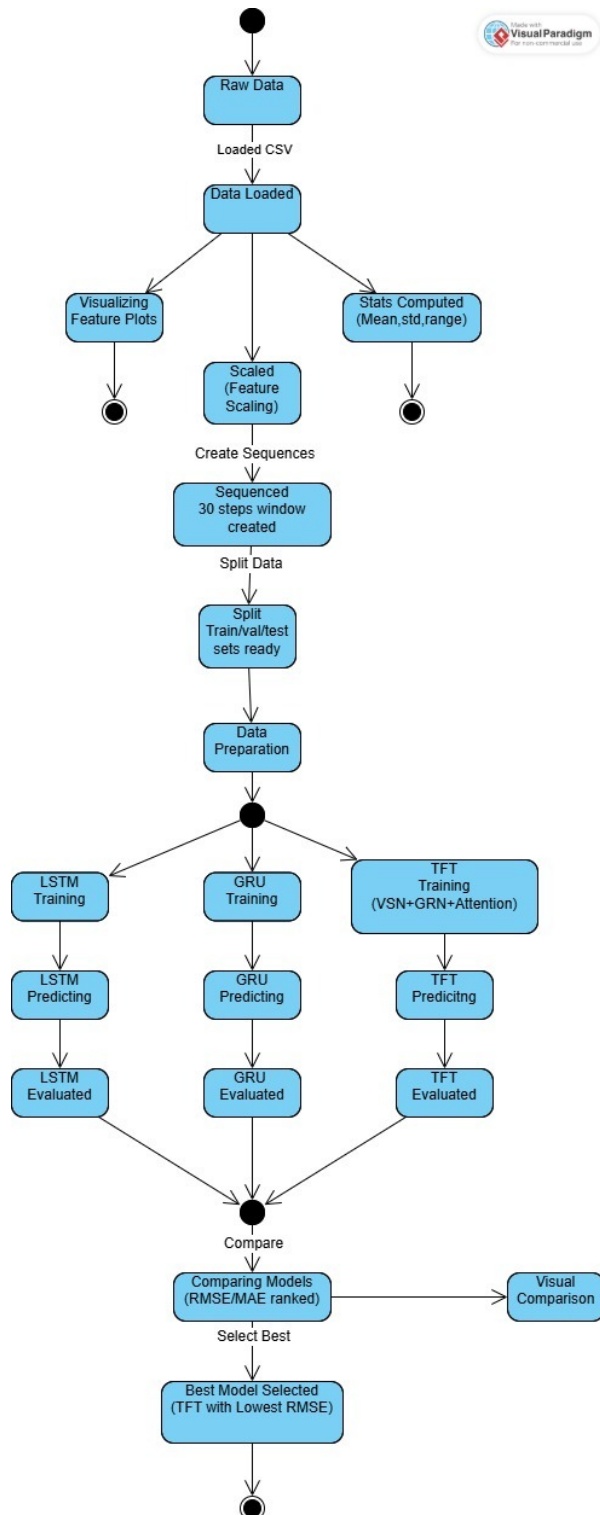


Figure 4.3: State Transition

4.4 USE CASE

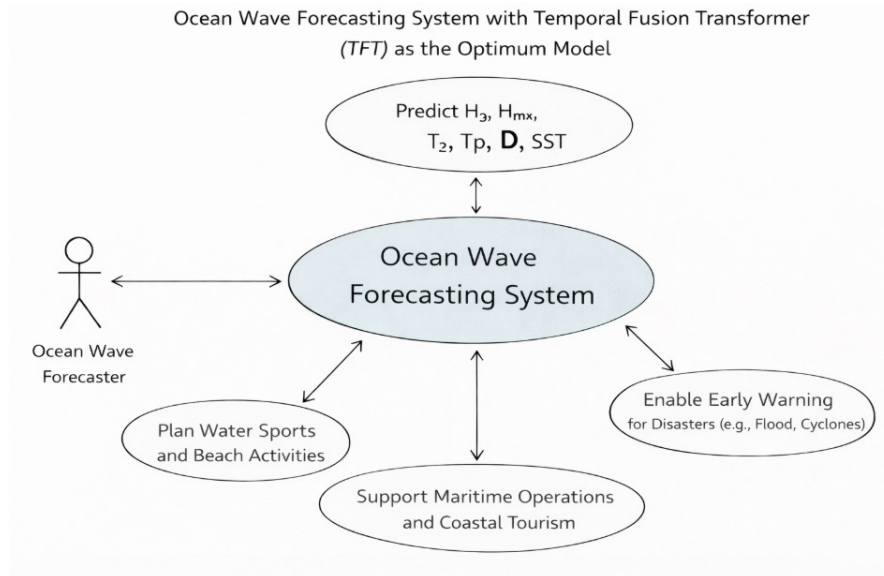


Figure 4.4: Use Case Diagram

4.5 CLASS DIAGRAM

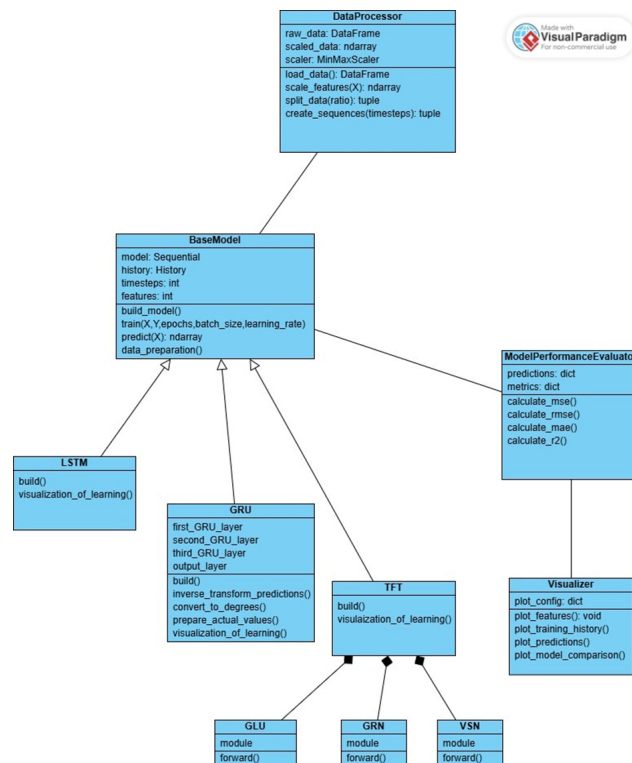


Figure 4.5: Class Diagram

4.6 DATA FLOW DIAGRAM

4.6.1 Data FLOW 0

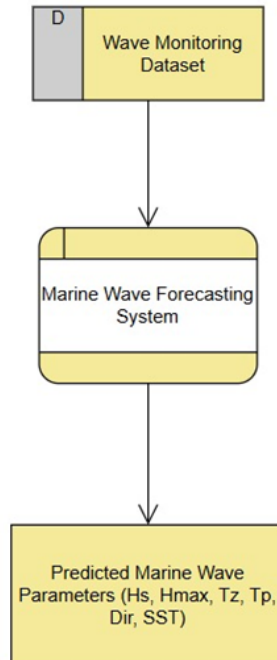


Figure 4.6: Data FLOW 0

4.6.2 Data FLOW 1

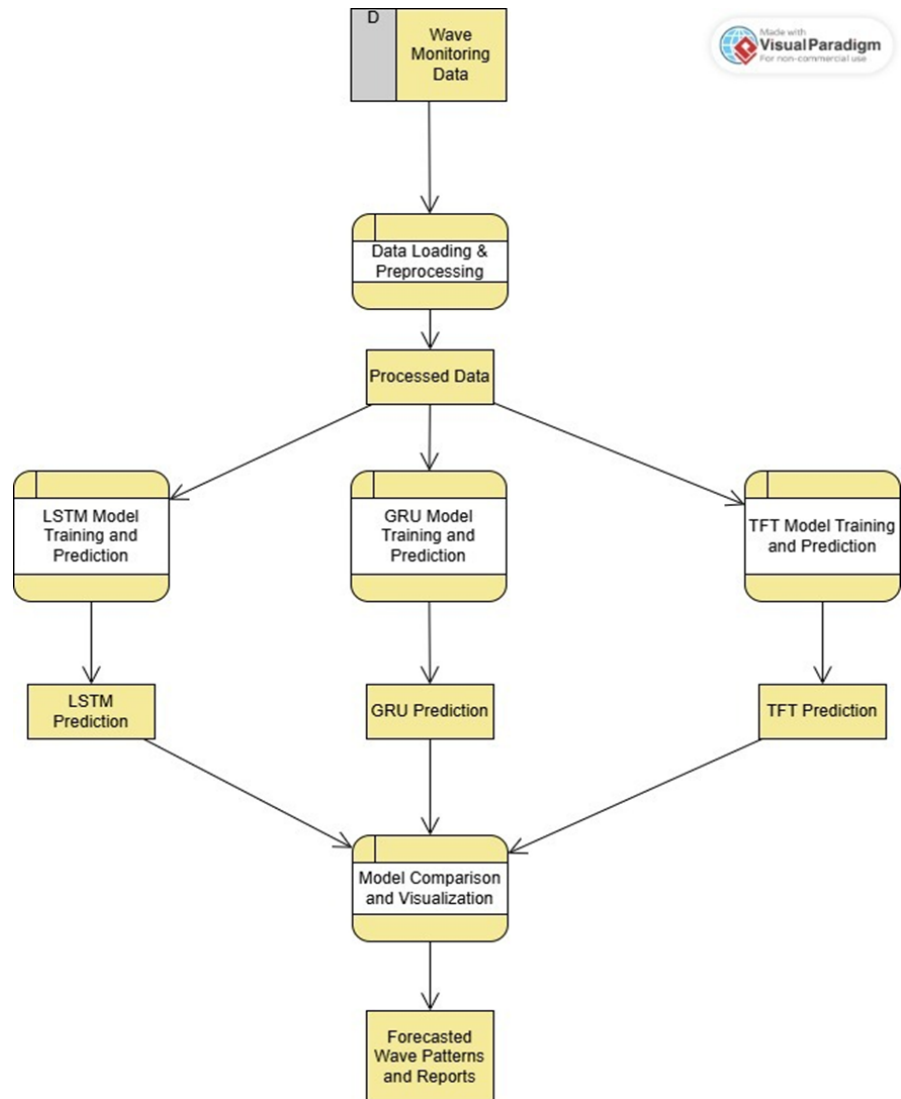


Figure 4.7: Data FLOW 1

4.6.3 Data FLOW 2

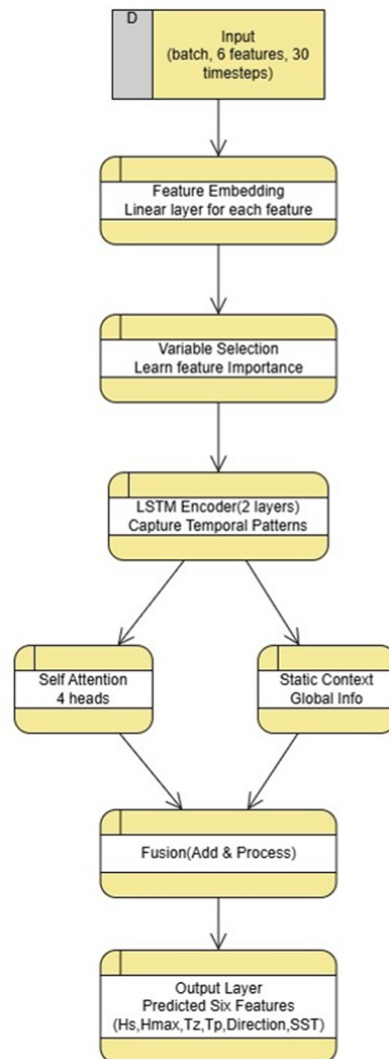


Figure 4.8: Data FLOW 2

CHAPTER 5

METHODOLOGY

5.1 SITES SELECTION AND DESCRIPTION

Australia's eastern coast is home to the suburb of Mooloolaba, which has been shown in the map in Figure 5.1. Mooloolaba was chosen as the study site because of the dynamic nature of its coastal environment and also due to the availability of quality data and buoy observations taken over a long period of time. It is situated on the Sunshine Coast of Queensland. The region experiences remarkable wave activity which is influenced by seasonal weather conditions, cyclonic events, and swell propagation from the Pacific Ocean. The presence of active marine operations, recreational water activities (swimming, water sports and diving), coastal tourism, and commercial fishing makes weather forecasting important in this area.

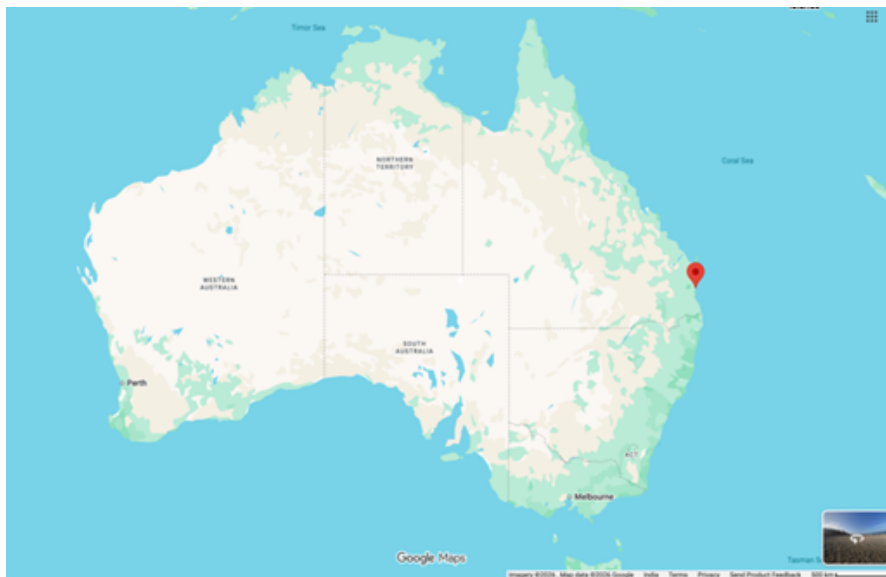


Figure 5.1: Location of Mooloolaba in Australia map

Mooloolaba experiences diverse sea states, making it suitable for testing the robustness of forecasting models. Also, the buoy anchored at Mooloolaba gives continuous and standardized measurements. The data obtained is thus reliable and of high quality. Lastly, this area supports tourism, marine transport, fisheries, and coastal management activities making it the area of significant economic importance.

Accurate predictions generated by models play a huge role in designing robust

coastal infrastructure and also in erosion monitoring. Reliable and quick forecasts about the state of the sea help in the planning of water sports, boating and other beach activities. Also, early identification of abnormal wave conditions supports warning systems during floods, cyclones or any other potential disaster.

5.2 DATASET ACQUISITION AND PREPARATION

The wave dataset used in this study was obtained from the Mooloolaba Coastal Wave Dataset available through the Queensland Government Open Data Portal. The dataset consists of certain measured and derived ocean wave parameters such as Significant Sea Surface Temperature (SST), Wave Direction (relative to true north), Peak Energy Wave Period (T_p), Zero Upcrossing Wave Period (T_z), Maximum Wave Height (H_{max}), Significant Wave Height (H_s) recorded at 30-minute intervals using an offshore wave buoy monitoring system.

In Figure 5.2, the acquired data features have been visualized. The data collected from Mooloolaba was preprocessed and structured into multivariate time-series format to make one-step ahead forecasting. It was done using three deep learning architectures namely, Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), and Temporal Fusion Transformer (TFT).

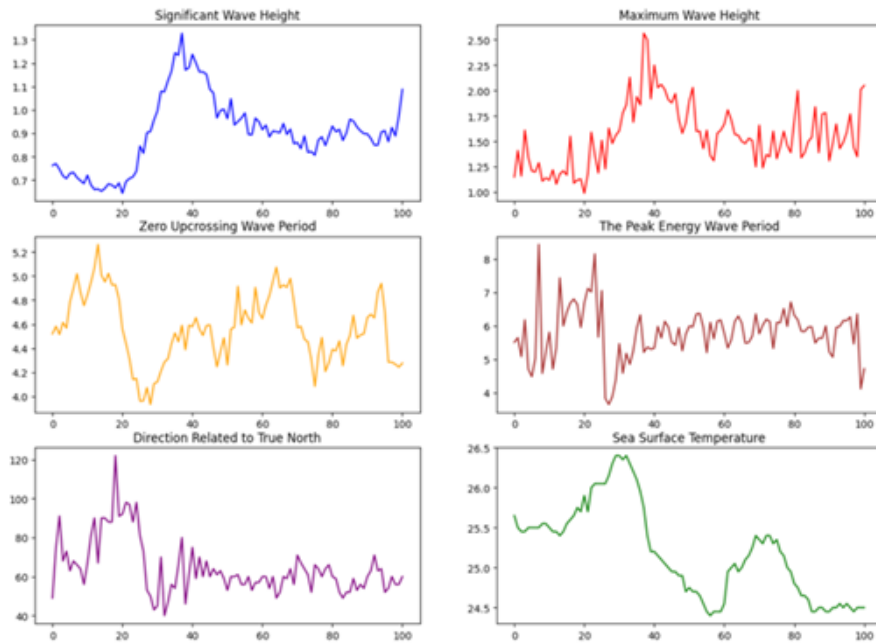


Figure 5.2: Visualisation of the features.

The data has been loaded into a Pandas DataFrame for manipulation. In the raw

dataset, missing and corrupted reading values were carefully handled by replacing them with -99.90. Later on, all instances of -99.90 were replaced with the standard NumPy NaN (Not a Number) value. Afterward, multivariate records with missing values were eliminated through deletion strategies. Table 5.1 reflects the statistical summary of the data after all these pre-processing stages.

Table 5.1: Statistical Summary of the Dataset

Stat	Hs	Hmax	Tz	Tp	Peak Direction	SST
count	43454.00	43454.00	43454.00	43454.00	43454.00	43454.00
mean	1.237799	2.090125	5.619685	9.011972	98.626594	23.949641
std	0.528608	0.897640	0.928533	2.390107	24.275165	2.231022
min	0.294000	0.510000	3.076000	2.720000	5.000000	19.800000
25%	0.839000	1.410000	4.981000	7.292000	85.000000	21.900000
50%	1.130000	1.900000	5.530000	8.886000	101.000000	23.950000
75%	1.544000	2.600000	6.166000	10.677000	116.000000	26.050000
max	4.257000	7.906000	10.921000	21.121000	358.000000	28.650000

A neural network’s performance is not optimal when feature scales differ significantly Ioffe and Szegedy (2015); Pinheiro (2025). Feature normalization was applied and features were scaled to a bounded range $[0, 1]$ using Min-Max scaling. The normalized data was partitioned into discrete sequences using a sliding window technique. The dataset was partitioned into training and testing sets to prevent data leakage and provide sufficient test samples for validation Feng et al. (2023).

5.3 MODEL IMPLEMENTATION

5.3.1 LSTM

Long Short-Term Memory (LSTM) networks reduce the vanishing gradient problem Hochreiter and Schmidhuber (1997), which is a very common problem in traditional Recurrent Neural Networks (RNNs) as LSTM utilizes a dedicated cell state to maintain long-term information Bengio et al. (1994). The dedicated cell has following internal gates:

1. **Forget Gate:** It removes irrelevant data.
2. **Input Gate:** It is for updating of state with new appropriate features.
3. **Output Gate:** It determines the state for subsequent time steps.

This architecture allows the model to bridge large temporal gaps efficiently and also helps to maintain a stable gradient flow during training.

The LSTM network was used to capture the worldly dependencies in ocean wave data. After the data processing, the model was trained on all six parameters. The model contains three consecutive LSTM layers with 32, 16, and 10 hidden units, respectively. An integration of a dropout layer having rate of 0.2 between second and third layers of LSTM was done to reduce the overfitting risk. The final output is generated through a fully connected dense layer with six neurons, which enables the simultaneous prediction of all target wave marine parameters.

Adam optimizer was employed with learning rate = 0.001, and Mean Squared Error (MSE) loss for regression. Maximum epochs were set to 15 with batch size 200. The model was trained with a validation split of 25%, which allows us to monitor the generalized performance in real-time and also secures that the network learned the fundamental patterns of the maritime environment successfully. Figure 6.5 represents the predicted and Real data are plotted using LSTM model.

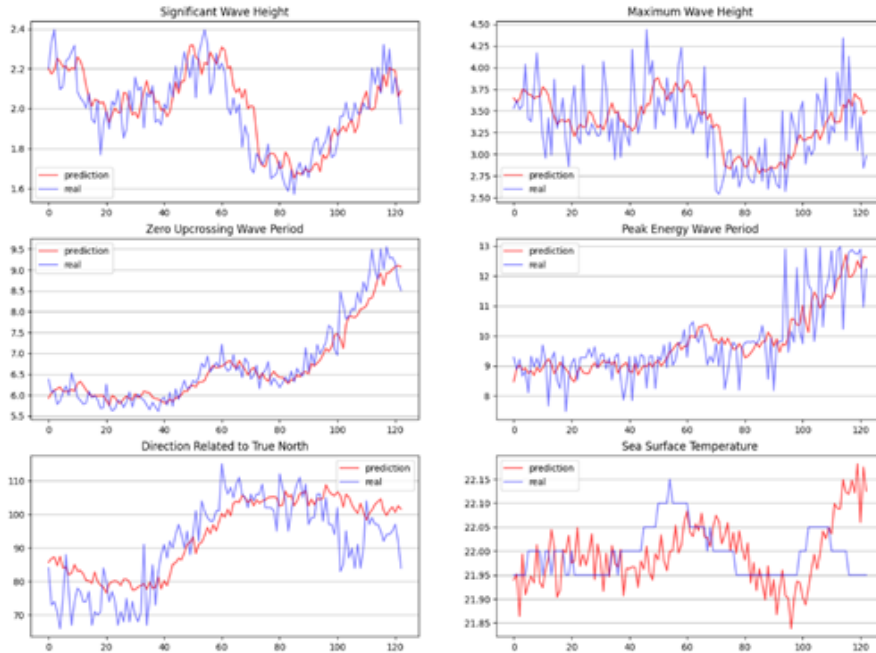


Figure 5.3: Visualization of the Real and Predicted Values with LSTM model

5.3.2 GRU

Gated Recurrent Unit (GRU) is a type of the recurrent neural network (RNN) algorithmic architecture Cho et al. (2014) to reduce the complexity of LSTM network. GRU

mitigates the vanishing gradient problem of traditional RNNs still maintain the computational efficiency by reducing the number of getting mechanisms. While LSTM employs input, forget and output, total 3 gates, GRU have two gates, first is the reset gate and the other is the update gate. It makes GRU computationally less expensive and achieve comparable performance in many sequence modeling tasks Chung et al. (2014).

The update gate controls state retention and the reset gate determines information forgetting which enables effective long-term dependency capture Chung et al. (2014).

The proposed GRU architecture consists of a deep stacked configuration with three GRU layers followed by dense layers for multi-output regression. The three-layer stacked GRU configuration of (128 to 64 to 32) is for hierarchical feature extraction, where the first two layers use `return_sequences=True` to pass the complete sequence to subsequent layers, while the final GRU layer returns only the last hidden state for prediction Pascanu et al. (2014). The batch normalization layers are implemented to normalize activations, minimize internal covariate shift, and to adding speed in training convergence Ioffe and Szegedy (2015). Elastic net regularization was applied where $\lambda_1 - \lambda_2 = 0.001$ on both kernel and recurrent weights to avoid overfitting by implementing penalty on large weight magnitudes Zou and Hastie (2005), and progressive dropout rates (30%, 30%, 20%, 20%) has been applied to improve generalization and prevention of co-adaptation of neurons Srivastava et al. (2014). Two fully connected layers with ReLU activation functions serve as a non-linear transformation stage before the output layer, which enable the model to learn complex mappings between GRU representations and target variables. A linear dense layer with 7 units produces simultaneous predictions for all wave parameters, making it as multi-target regression.

Adam optimizer was employed with learning rate = 0.0005, clipnorm = 1.0, which combines AdaGrad’s sparse gradient handling with RMSProp’s non-stationary objective management Kingma and Ba (2015), and Mean Squared Error (MSE) loss for regression. Three callback mechanisms were implemented to optimize training. Early stopping (patience = 15 epochs), learning rate reduction (factor 0.5 after 5 plateaued epochs), and model checkpointing were few of the callback mechanisms. Maximum epochs were set to 50 with batch size 128.

After prediction, by inverse scaling original values were restored, and direction was

reconstructed from sine/cosine components with four quadrant arctangent function:

$$\theta = \left(\text{atan2}(\sin \theta, \cos \theta) \cdot \frac{180}{\pi} \right) \bmod 360 \quad (5.1)$$

This maintains the predicted direction within $[0^\circ, 360^\circ]$ and keep the circular nature of directional data. Figure 6.8 represents the predicted and Real data are plotted using GRU model.

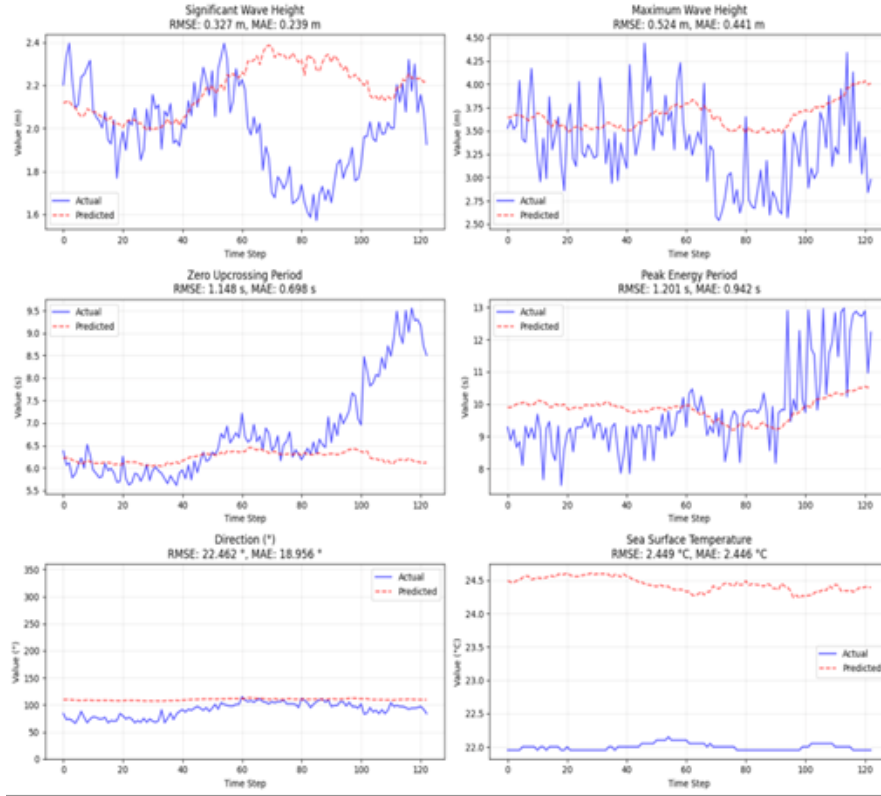


Figure 5.4: Visualization of the Real and Predicted Values with GRU model

5.3.3 TFT

Temporal Fusion Transformer(TFT) was implemented for multivariate time series forecasting integrating recurrent networks, attention mechanisms and feature selection modules.

Firstly, the basic building blocks of Temporal Fusion Transformer were defined Lim et al. (2021):

1. **Gated Linear Unit (GLU):** It controls information flow and implements a sigmoid gating mechanism Dauphin et al. (2017).

2. **Gated Residual Network (GRN):** It combines attention output and integrates static context and implements a gating mechanism that decides how much new information to allow and filters noise Lim et al. (2021).
3. **Variable Selection Network (VSN):** It dynamically selects features which are important at each timestep and suppresses less important features Lim et al. (2021).
4. **Interpretable Multi-Head Attention:** It captures long-term temporal dependencies Lim et al. (2021); Vaswani et al. (2017).

The model receives the tensor shape input having shape (batch, features, timesteps), which is first rearranged into temporal format (batch, timesteps, features), following the sequence modeling framework described in Lim et al. (2021). At every timestep, each feature is projected independently into a shared hidden representation through linear embedding layers Lim et al. (2021). These embedded features are then processed based on timesteps by the Variable Selection Network, producing a refined sequence of feature representations that are temporally aligned Lim et al. (2021). The sequence obtained is then passed onto a multi-layer LSTM encoder to capture temporal dependencies across the 30-step historical window Hochreiter and Schmidhuber (1997). Parallel to the LSTM encoder, the full LSTM output sequence is processed through multi-head self-attention to identify important long-term temporal patterns Vaswani et al. (2017), and the final attended representation is refined through the gating network Lim et al. (2021). The output produced is combined with a static context vector and further transformed using a feed-forward network Lim et al. (2021).

Having defined the model, data preparation for TFT is carried out. The multivariate time-series data was transformed using a window size of 30 timestep to generate input-output pairs for one-step-ahead multitarget forecasting Lim et al. (2021).

The TFT model was trained using processed multivariate time-series data for 50 epochs with Mean Squared Error (MSE) as the loss function (Bishop, 2006) and Adam optimizer (learning rate = 0.001) Kingma and Ba (2015). A ReduceLROnPlateau scheduler was applied to lessen the learning rate based on validation loss, and gradient clipping was carried out to stabilize training Pascanu et al. (2013) can be inferred from Figure 6.10. Model performance was monitored on the validation set with early stop-

ping implemented to prevent overfitting Prechelt (1998).

The performance of the TFT model was analysed further, represented in Figure 6.11.

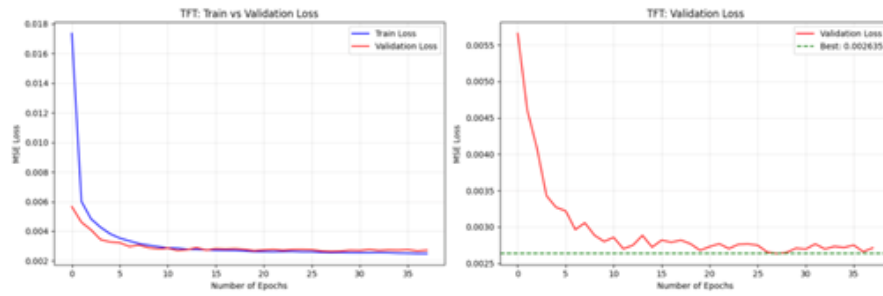


Figure 5.5: TFT learning curves

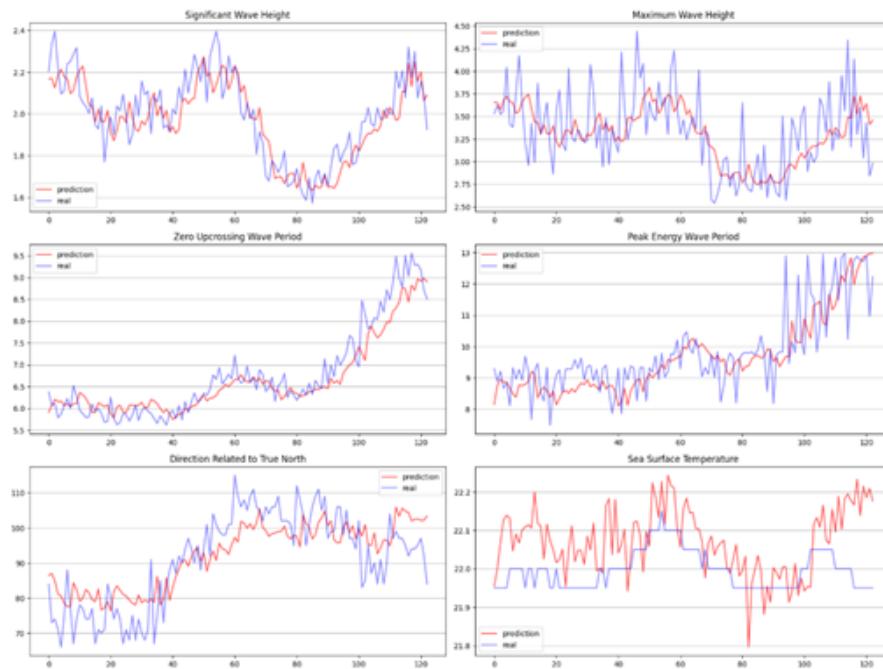


Figure 5.6: Visualization of the Real and Predicted values with TFT

CHAPTER 6

PROJECT IMPLEMENTATION

	Date/Time	Hs	Hmax	Tz	Tp	Peak Direction	SST
0	01/01/2017 00:00	-99.900	-99.90	-99.900	-99.900	-99.9	-99.90
1	01/01/2017 00:30	0.875	1.39	4.421	4.506	-99.9	-99.90
2	01/01/2017 01:00	0.763	1.15	4.520	5.513	49.0	25.65
3	01/01/2017 01:30	0.770	1.41	4.582	5.647	75.0	25.50
4	01/01/2017 02:00	0.747	1.16	4.515	5.083	91.0	25.45

Figure 6.1: Loading Data

	Hs	Hmax	Tz	Tp	Peak Direction	SST
0	0.763	1.15	4.520	5.513	49.0	25.65
1	0.770	1.41	4.582	5.647	75.0	25.50
2	0.747	1.16	4.515	5.083	91.0	25.45
3	0.718	1.61	4.614	6.181	68.0	25.45
4	0.707	1.34	4.568	4.705	73.0	25.50

Figure 6.2: Deleting NaN

Epoch 1/15	158/158	9s	26ms/step	loss: 0.0211	mean_squared_error: 0.0211	val_loss: 0.0040	val_mean_squared_error: 0.0040
Epoch 2/15	158/158	2s	14ms/step	loss: 0.0060	mean_squared_error: 0.0060	val_loss: 0.0032	val_mean_squared_error: 0.0032
Epoch 3/15	158/158	2s	14ms/step	loss: 0.0050	mean_squared_error: 0.0050	val_loss: 0.0028	val_mean_squared_error: 0.0028
Epoch 4/15	158/158	3s	14ms/step	loss: 0.0043	mean_squared_error: 0.0043	val_loss: 0.0024	val_mean_squared_error: 0.0024
Epoch 5/15	158/158	2s	15ms/step	loss: 0.0037	mean_squared_error: 0.0037	val_loss: 0.0021	val_mean_squared_error: 0.0021
Epoch 6/15	158/158	4s	23ms/step	loss: 0.0034	mean_squared_error: 0.0034	val_loss: 0.0019	val_mean_squared_error: 0.0019
Epoch 7/15	158/158	2s	14ms/step	loss: 0.0031	mean_squared_error: 0.0031	val_loss: 0.0018	val_mean_squared_error: 0.0018
Epoch 8/15	158/158	3s	14ms/step	loss: 0.0029	mean_squared_error: 0.0029	val_loss: 0.0018	val_mean_squared_error: 0.0018
Epoch 9/15	158/158	2s	14ms/step	loss: 0.0027	mean_squared_error: 0.0027	val_loss: 0.0017	val_mean_squared_error: 0.0017
Epoch 10/15	158/158	3s	14ms/step	loss: 0.0026	mean_squared_error: 0.0026	val_loss: 0.0017	val_mean_squared_error: 0.0017
Epoch 11/15	158/158	4s	23ms/step	loss: 0.0026	mean_squared_error: 0.0026	val_loss: 0.0016	val_mean_squared_error: 0.0016
Epoch 12/15	158/158	2s	14ms/step	loss: 0.0025	mean_squared_error: 0.0025	val_loss: 0.0016	val_mean_squared_error: 0.0016
Epoch 13/15	158/158	2s	14ms/step	loss: 0.0024	mean_squared_error: 0.0024	val_loss: 0.0015	val_mean_squared_error: 0.0015
Epoch 14/15	158/158	2s	14ms/step	loss: 0.0023	mean_squared_error: 0.0023	val_loss: 0.0015	val_mean_squared_error: 0.0015
Epoch 15/15	158/158	2s	14ms/step	loss: 0.0023	mean_squared_error: 0.0023	val_loss: 0.0015	val_mean_squared_error: 0.0015

Figure 6.3: LSTM Implementation

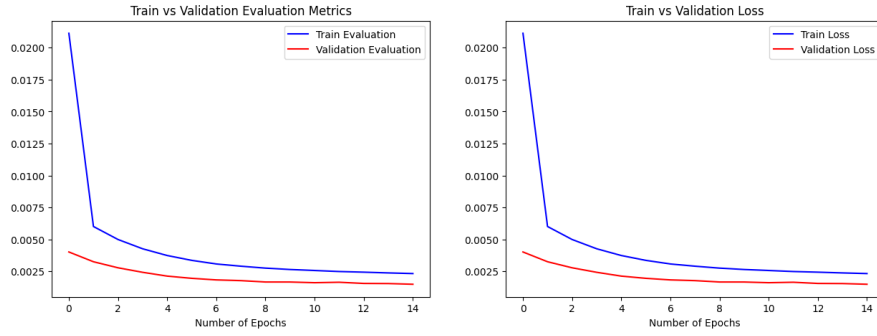


Figure 6.4: LSTM Learning Visualization

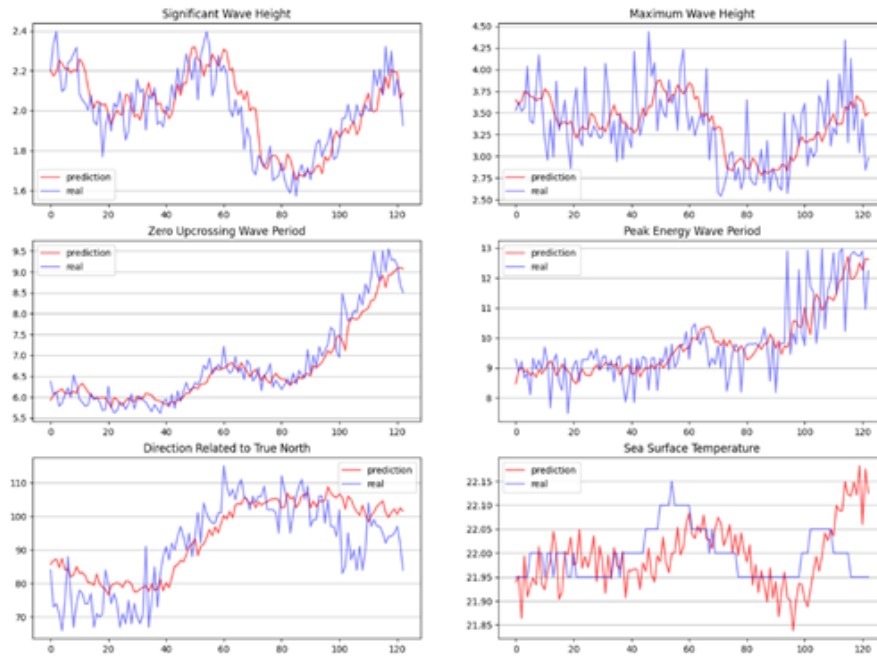


Figure 6.5: Visualization of the Real and Predicted Values with LSTM model

```
=====
PREPARING DATA
=====

CORRECTED GRU DATA PREPARATION
=====

1. Converting direction to sin/cos...
New columns: ['Hs', 'Hmax', 'Tz', 'Tp', 'Peak Direction_sin', 'Peak Direction_cos', 'SST']
Number of features: 7

2. Scaling data...
Scaled data shape: (43454, 7)

3. Splitting data...
Train shape: (42000, 7)
Test shape: (1454, 7)

4. Creating sequences (length=30)...
X_train shape: (41969, 7, 30)
y_train shape: (41969, 7)
X_test shape: (1423, 7, 30)
y_test shape: (1423, 7)
```

Figure 6.6: GRU Data Processing

Layer (type)	Output Shape	Param #
gru (GRU)	(None, 7, 128)	61,440
batch normalization (BatchNormalization)	(None, 7, 128)	512
dropout_1 (Dropout)	(None, 7, 128)	0
gru_1 (GRU)	(None, 7, 64)	37,248
batch normalization_1 (BatchNormalization)	(None, 7, 64)	256
dropout_2 (Dropout)	(None, 7, 64)	0
gru_2 (GRU)	(None, 32)	9,408
batch normalization_2 (BatchNormalization)	(None, 32)	128
dropout_3 (Dropout)	(None, 32)	0
dense_1 (Dense)	(None, 64)	2,112
batch normalization_3 (BatchNormalization)	(None, 64)	256
dropout_4 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
output_layer (Dense)	(None, 7)	231

Figure 6.7: GRU Implementation

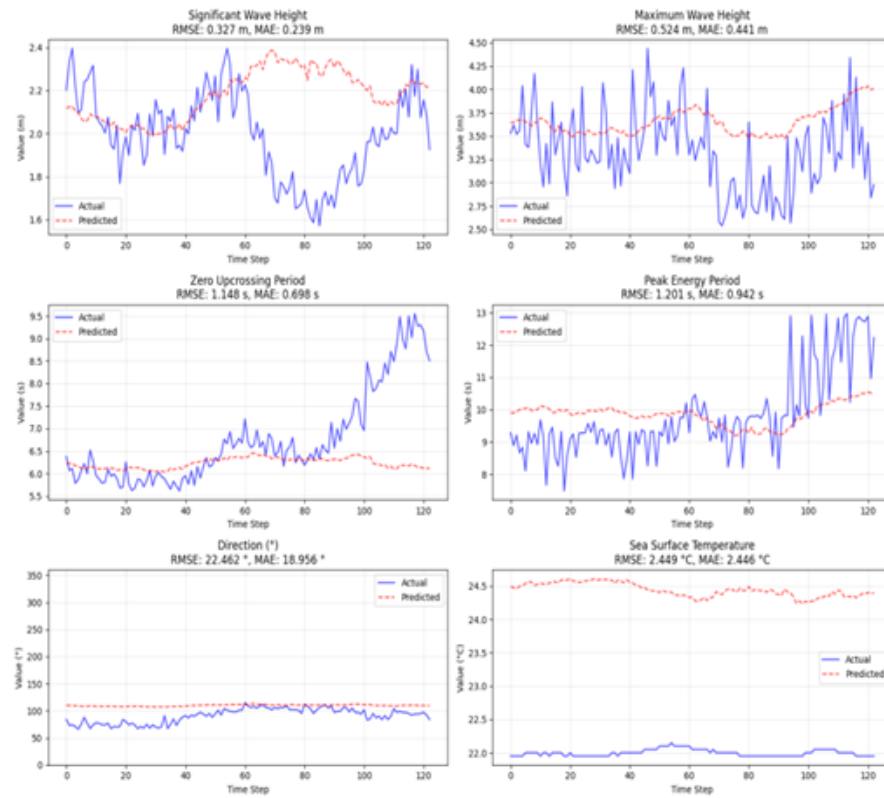


Figure 6.8: Visualization of the Real and Predicted Values with GRU model

```

Using device: cpu
Total TFT Parameters: 254,152

=====
TRAINING TFT MODEL
=====
Epoch 01/50 | Train Loss: 0.020834 | Val Loss: 0.009653
  ✓ Best model saved (Val Loss: 0.009653)
Epoch 02/50 | Train Loss: 0.007908 | Val Loss: 0.005915
  ✓ Best model saved (Val Loss: 0.005915)
Epoch 03/50 | Train Loss: 0.005772 | Val Loss: 0.005289
  ✓ Best model saved (Val Loss: 0.005289)
Epoch 04/50 | Train Loss: 0.004856 | Val Loss: 0.003966
  ✓ Best model saved (Val Loss: 0.003966)
Epoch 05/50 | Train Loss: 0.004286 | Val Loss: 0.003736
  ✓ Best model saved (Val Loss: 0.003736)
Epoch 06/50 | Train Loss: 0.003880 | Val Loss: 0.003427
  ✓ Best model saved (Val Loss: 0.003427)
Epoch 07/50 | Train Loss: 0.003609 | Val Loss: 0.003129
  ✓ Best model saved (Val Loss: 0.003129)
Epoch 08/50 | Train Loss: 0.003442 | Val Loss: 0.003190
Epoch 09/50 | Train Loss: 0.003316 | Val Loss: 0.002957
  ✓ Best model saved (Val Loss: 0.002957)
Epoch 10/50 | Train Loss: 0.003271 | Val Loss: 0.003212
Epoch 11/50 | Train Loss: 0.003201 | Val Loss: 0.002932
...
TRAINING COMPLETE
Best Validation Loss: 0.002697
=====
  ✓ Best model loaded

```

Figure 6.9: TFT Implementation

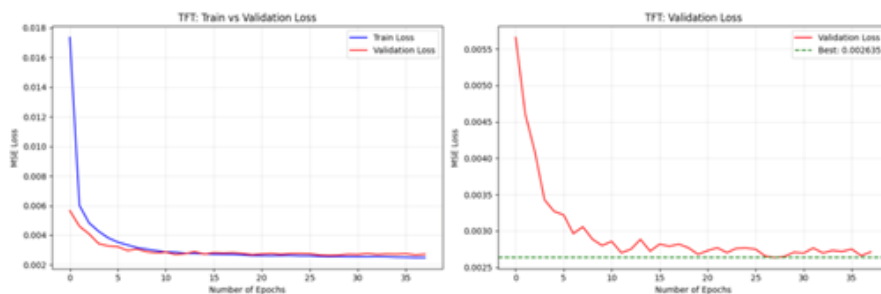


Figure 6.10: TFT Learning

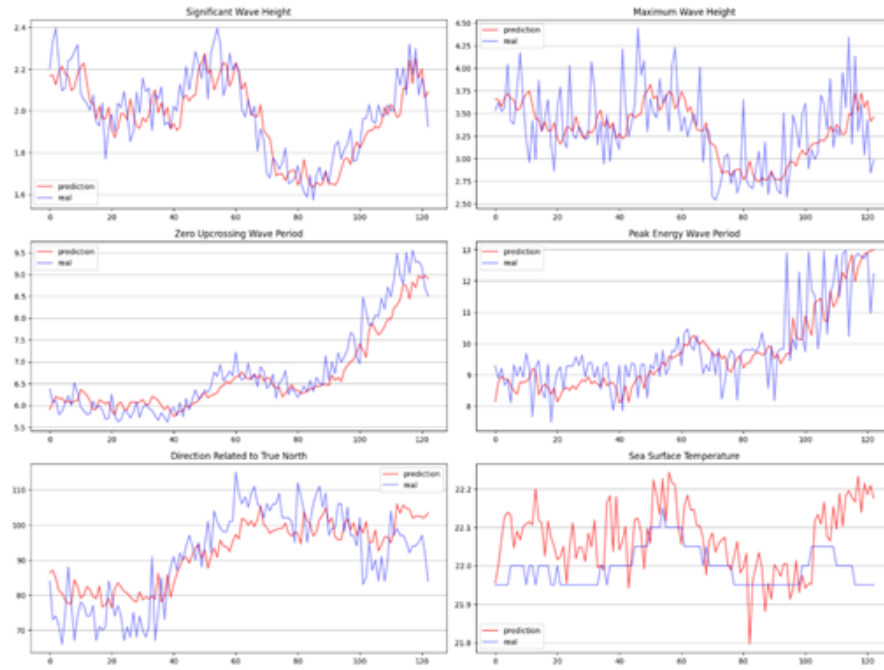


Figure 6.11: Visualization of the Real and Predicted values with TFT

CHAPTER 7

RESULTS AND DISCUSSION

The parameters namely Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), Mean Absolute Percentage Error (MAPE) and coefficient of determination (R^2) was used to evaluate performance across six target variables: Significant Wave Height (Hs), Maximum Wave Height (Hmax), Zero Upcrossing Wave Period (Tz), Peak Energy Wave Period (Tp), Peak Direction, and Sea Surface Temperature (SST).

Table 7.1: Model Comparison with RMSE

Feature	LSTM	GRU	TFT	Best Model
Hs	0.0889	0.9093	0.0824	TFT
Hmax	0.2484	1.3672	0.242	TFT
Tz	0.3416	1.6312	0.3194	TFT
Tp	1.9561	2.5959	1.9391	TFT
Direction	14.3529	18.6922	13.5743	TFT
SST	0.2117	2.0568	0.1148	TFT
Overall	5.917	7.8073	5.6006	TFT

From Table 7.1, it can be observed that TFT has resulted in better performance than both LSTM and GRU across all six parameters. For Significant Wave Height, TFT performed better in terms of RMSE with 0.0824 m, which was slightly better than LSTM (0.0889 m) and significantly better than GRU (0.9093 m). The largest improvement was observed in SST prediction, where TFT performed 45.8% better than LSTM and 94.4% better than GRU. Peak Direction is complex to calculate, due to its inherently cyclic nature, achieved RMSE values in favour of TFT.

Table 7.2: Model Comparison with MAE

Feature	LSTM	GRU	TFT	Best Model
Hs	0.0673	0.8256	0.0611	TFT
Hmax	0.182	1.238	0.1723	TFT
Tz	0.2595	1.4075	0.2365	TFT
Tp	1.4329	2.1708	1.4345	LSTM
Direction	9.1924	12.4436	8.7315	TFT
SST	0.1528	1.9832	0.0893	TFT
Overall	1.8812	3.3448	1.7875	TFT

The Mean Absolute Error (MAE) analysis was also performed across all three models which can be observed in Table 7.2. It is measured using the average absolute

deviation without the squared penalty of RMSE. Here also, TFT achieved substantially better results across most of the target values. LSTM marginally outperformed TFT for Peak Energy Wave Period, where MAE was calculated as 1.4329 s for LSTM against 1.4345 s for TFT, making it the only instance where TFT did not achieve the best score.

Table 7.3: Model Comparison with MAPE

Feature	LSTM	GRU	TFT	Best Model
Hs	6.5054	34.0486	6.3389	TFT
Hmax	10.0863	29.0558	9.4453	TFT
Tz	4.8094	12.6649	4.5102	TFT
Tp	17.5934	26.0057	17.7032	LSTM
Direction	10.6545	18.3181	9.826	TFT
SST	0.6086	5.6447	0.3608	TFT
Overall	8.3763	20.9563	8.0307	TFT

Along with MAE, Mean Absolute Percentage Error (MAPE) analysis was also performed across all three models which can be observed in Table 7.3. MAPE calculates the average error in percentage which is helpful to interpret across different scale. MAPE also achieved a similar result to MAE with TFT overall the best performing model.

Further information on the additional insights into the capabilities of the models can be understood from the R^2 score. R^2 score of TFT was calculated as 0.9797, which was the highest, followed by LSTM (0.9774) and GRU (0.9606), which points out that although all models successfully captured underlying variance in ocean wave dynamics, TFT has provided the best predictions.

Graphical representation, as in Figure 7.1, also reveals that while LSTM and TFT prediction paths stay closer to the actual track, GRU exhibits larger deviation, particularly during rapid fluctuations. This supports the quantitative measurements that attest to TFT's superior ability to identify fine-grained temporal variations.

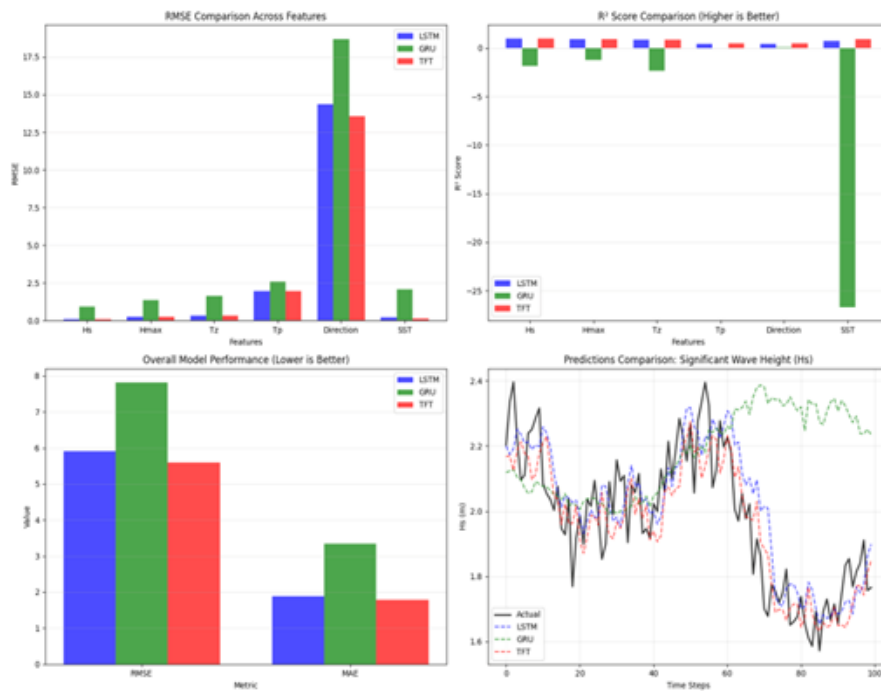


Figure 7.1: Model Performance Comparison: LSTM vs GRU vs TFT

CHAPTER 8

CONCLUSION AND FURTHER ENHANCEMENTS

This research work has provided a detailed comparative analysis of three deep learning models: Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), and Temporal Fusion Transformer (TFT) for multi-parameter ocean wave prediction. The models were tested on a dataset of over 43,000 observations of six oceanographic parameters: Significant Wave Height (Hs), Maximum Wave Height (Hmax), Zero Upcrossing Wave Period (Tz), Peak Energy Wave Period (Tp), Peak Direction, and Sea Surface Temperature (SST).

TFT achieves its better performance from its attention-based architecture, which focuses on important time steps through Variable Selection Networks and Multi-Head Attention mechanisms. Instead of sequential processing in LSTM and GRU, TFT's transformer architecture allows direct modeling of long-range dependencies without information degradation. GRU's lower performance and LSTM's competitive second-place ranking demonstrate the better possibilities of usage of the transformer architecture model in several scenarios, which might help achieve reliable performance of the model.

These findings help to draw some practical understandings also. Though TFT provides better performance, its computational complexity against deployment constraints should also be considered. In a situation where resources are limited, LSTM may give a balanced result of efficiency and accuracy. Therefore, the adoption of the appropriate model should be based on situational necessity. In future research, extending the prediction horizon to multi-step forecasting, adding other meteorological variable in the model, addressing special maritime situation like cyclone, earthquake will further increase reliability of the model.

BIBLIOGRAPHY

- Adytia, D., Saepudin, D., Tarwidi, D., Pudjaprasetya, S. R., Husrin, S., Sopaheluwakan, A., and Prasetya, G. (2023). Modelling of deep learning-based downscaling for wave forecasting in coastal area. *Water*, 15(1):204.
- Afzal, M. S., Kumar, L., Chugh, V., Kumar, Y., and Zuhair, M. (2023). Prediction of significant wave height using machine learning and its application to extreme wave analysis. *Journal of Earth System Science*, 132(51).
- Bengio, Y., Simard, P., and Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166.
- Booij, N., Ris, R. C., and Holthuijsen, L. H. (1999). A third-generation wave model for coastal regions. *Journal of Geophysical Research: Oceans*, 104(C4):7649–7666.
- Cho, K., Van Merriënboer, B., Gulcehre, C., et al. (2014). Learning phrase representations using rnn encoder-decoder. In *EMNLP*.
- Chung, J., Gulcehre, C., Cho, K., and Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks.
- Dai, X. and Liang, Y. (2025). Sea surface temperature prediction using convlstm. *EGU-sphere*.
- Dauphin, Y. N., Fan, A., Auli, M., and Grangier, D. (2017). Language modeling with gated convolutional networks. In *ICML*, pages 933–941.
- Feng, H., Wang, Y., Li, Z., Zhang, N., Zhang, Y., and Gao, Y. (2023). Information leakage in deep learning-based hyperspectral image classification. *Remote Sensing*, 15(15):3793.
- Han, J., Kamber, M., and Pei, J. (2012). *Data Mining: Concepts and Techniques*. Morgan Kaufmann.
- Hao, P., Li, S., and Gao, Y. (2023). Significant wave height prediction based on deep learning in the south china sea. *Frontiers in Marine Science*, 9.

- Hasselmann, S., Hasselmann, K., Allender, J. H., and Barnett, T. P. (1988). Nonlinear energy transfer in a gravity-wave spectrum. *Journal of Physical Oceanography*.
- Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780.
- Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training.
- James, S., Zhang, Y., and O’Donncha, F. (2018). A machine learning framework to forecast wave conditions. *Coastal Engineering*, 137:1–10.
- Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *ICLR*.
- Lawal, Z. K., Yassin, H., Lai, D. T. C., and Idris, A. C. (2024). Modeling the complex spatio-temporal dynamics of ocean wave parameters. *Ocean Engineering*.
- LeCun, Y., Bengio, Y., and Hinton, G. (2015). Deep learning. *Nature*, 521:436–444.
- Lim, B., Arik, S. O., Loeff, N., and Pfister, T. (2021). Temporal fusion transformers for time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764.
- Pascanu, R., Gulcehre, C., Cho, K., and Bengio, Y. (2014). How to construct deep recurrent neural networks.
- Pascanu, R., Mikolov, T., and Bengio, Y. (2013). On the difficulty of training recurrent neural networks. In *ICML*, pages 1310–1318.
- Pinheiro, J. M. H. (2025). Impact of feature scaling in machine learning.
- Prechelt, L. (1998). Early stopping—but when? In *Neural Networks: Tricks of the Trade*. Springer.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A simple way to prevent overfitting. *JMLR*, 15:1929–1958.
- Vaswani, A. et al. (2017). Attention is all you need. In *NeurIPS*, pages 5998–6008.
- Zou, H. and Hastie, T. (2005). Regularization and variable selection via the elastic net. *JRSS-B*, 67(2):301–320.

Appendix A- Sample Code

```
/* Creating the LSTM Model*/

from keras.models import Sequential
from keras.layers import Dense, Dropout, LSTM, Input
from keras.optimizers import Adam

def lstm_model(x_data, y_data, num_epochs, batch_size, learning_rate):
    model = Sequential()
    model.add(Input(shape=(x_data.shape[1], x_data.shape[2])))
    model.add(LSTM(32, return_sequences=True))
    model.add(LSTM(16, return_sequences=True))
    model.add(Dropout(0.2))
    model.add(LSTM(10))
    model.add(Dense(6))

    optimizer = Adam(learning_rate=learning_rate)
    model.compile(loss='mean_squared_error', optimizer=optimizer, metrics=['mean_squared_error'])

    history = model.fit(x_data, y_data, validation_split=0.25, epochs=num_epochs, batch_size=batch_size)
    return model, history

/* Visualization of the Real and Predicted Values*/

plt.figure(figsize=(17,25))

plt.subplot(6,2,1)
plt.plot(test_prediction[1300:,0], color='red', alpha=0.7, label='prediction')
plt.plot(y_test[1300:,0], color='blue', alpha=0.5, label='real')
plt.title('Significant Wave Height')
plt.legend()
plt.grid(True, axis='y')

plt.subplot(6,2,2)
plt.plot(test_prediction[1300:,1], color='red', alpha=0.7, label='prediction')
plt.plot(y_test[1300:,1], color='blue', alpha=0.5, label='real')
plt.title('Maximum Wave Height')
plt.legend()
plt.grid(True, axis='y')

plt.subplot(6,2,3)
plt.plot(test_prediction[1300:,2], color='red', alpha=0.7, label='prediction')
```

```

plt.plot(y_test[1300:,2], color='blue', alpha=0.5, label='real')
plt.title('Zero Upcrossing Wave Period')
plt.legend()
plt.grid(True, axis='y')

plt.subplot(6,2,4)
plt.plot(test_prediction[1300:,3], color='red', alpha=0.7, label='prediction')
plt.plot(y_test[1300:,3], color='blue', alpha=0.5, label='real')
plt.title('Peak Energy Wave Period')
plt.legend()
plt.grid(True, axis='y')

plt.subplot(6,2,5)
plt.plot(test_prediction[1300:,4], color='red', alpha=0.7, label='prediction')
plt.plot(y_test[1300:,4], color='blue', alpha=0.5, label='real')
plt.title('Direction Related to True North')
plt.legend()
plt.grid(True, axis='y')

plt.subplot(6,2,6)
plt.plot(test_prediction[1300:,5], color='red', alpha=0.7, label='prediction')
plt.plot(y_test[1300:,5], color='blue', alpha=0.5, label='real')
plt.title('Sea Surface Temperature')
plt.legend()
plt.grid(True, axis='y')
plt.show();

/* Data Preparation for GRU */

import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

def prepare_data_gru_fixed(df_original, sequence_length=30):
    """
    CORRECTED: Prepare data for GRU with proper directional feature handling
    """
    print("="*60)
    print("CORRECTED GRU DATA PREPARATION")
    print("="*60)

    # 1. Create copy
    df = df_original.copy()

    # 2. Convert direction to radians and then to sin/cos

```

```

print("\n1. Converting direction to sin/cos...")
direction_rad = np.radians(df['Peak Direction'])
df['Peak Direction_sin'] = np.sin(direction_rad)
df['Peak Direction_cos'] = np.cos(direction_rad)

# Reorder columns to: Hs, Hmax, Tz, Tp, Dir_sin, Dir_cos, SST
columns_order = ['Hs', 'Hmax', 'Tz', 'Tp', 'Peak Direction_sin', 'Peak Direction_cos', 'SST']
df = df[columns_order]

print(f"New columns: {df.columns.tolist()}")
print(f"Number of features: {len(df.columns)}")

# 3. Scale data (0 to 1 range)
print("\n2. Scaling data...")
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(df)

print(f"Scaled data shape: {scaled_data.shape}")

# 4. Train/Test split (same as LSTM: 42000 train, rest test)
print("\n3. Splitting data...")
train_data = scaled_data[:42000]
test_data = scaled_data[42000:]

print(f"Train shape: {train_data.shape}")
print(f"Test shape: {test_data.shape}")

# 5. Create sequences
print(f"\n4. Creating sequences (length={sequence_length})...")

def create_sequences(data):
    X, y = [], []
    for i in range(len(data) - sequence_length - 1):
        X.append(data[i:i + sequence_length])
        y.append(data[i + sequence_length + 1])

    X = np.array(X)
    y = np.array(y)

    # Reshape to (samples, features, timesteps)
    X = np.transpose(X, (0, 2, 1))

    return X, y

X_train, y_train = create_sequences(train_data)
X_test, y_test = create_sequences(test_data)

print(f"X_train shape: {X_train.shape}")
print(f"y_train shape: {y_train.shape}")

```

```

        print(f"X_test shape: {X_test.shape}")
        print(f"y_test shape: {y_test.shape}")

    return X_train, y_train, X_test, y_test, scaler, columns_order

# Prepare data
print("\n" + "="*60)
print("PREPARING DATA")
print("="*60)
X_train_gru, y_train_gru, X_test_gru, y_test_gru, scaler_gru, feature_names = prepare_data_g

/*GRU Model Implementation*/

from keras.layers import GRU, Input, Dense, Dropout, BatchNormalization
from keras.models import Sequential
from keras.regularizers import l1_l2
from keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint
from keras.optimizers import Adam

def fixed_gru_model(x_train, y_train, x_val, y_val, num_epochs=100, batch_size=64, learning_
    """
    FIXED GRU model implementation
    """
    print("\n" + "="*60)
    print("BUILDING GRU MODEL")
    print("="*60)

    n_features = x_train.shape[1] # Should be 7
    n_timesteps = x_train.shape[2] # Should be 30

    print(f"Input shape: ({n_timesteps} timesteps      {n_features} features)")
    print(f"Output shape: {n_features} features")

    model = Sequential()

    # Input layer
    model.add(Input(shape=(n_features, n_timesteps)))

    # First GRU layer
    model.add(GRU(128,
                    return_sequences=True,
                    kernel_regularizer=l1_l2(l1=0.001, l2=0.001),
                    recurrent_regularizer=l1_l2(l1=0.001, l2=0.001)))

```



```

model.add(BatchNormalization())
model.add(Dropout(0.3)) # Increased dropout

# Second GRU layer
model.add(GRU(64,
              return_sequences=True,
              kernel_regularizer=l1_l2(l1=0.001, l2=0.001),
              recurrent_regularizer=l1_l2(l1=0.001, l2=0.001)))
model.add(BatchNormalization())
model.add(Dropout(0.3))

# Third GRU layer
model.add(GRU(32,
              return_sequences=False,
              kernel_regularizer=l1_l2(l1=0.001, l2=0.001)))
model.add(BatchNormalization())
model.add(Dropout(0.2))

# Dense layers with activation
model.add(Dense(64, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.2))

model.add(Dense(32, activation='relu'))

# Output layer - CRITICAL: Must match number of features (7)
model.add(Dense(n_features, name='output_layer'))

# Compile with adjusted learning rate
optimizer = Adam(learning_rate=learning_rate, clipnorm=1.0)
model.compile(
    loss='mse',
    optimizer=optimizer,
    metrics=['mse', 'mae']
)

# Callbacks
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=15,
    restore_best_weights=True,
    min_delta=0.0001,
    verbose=1
)

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=5,

```

```

        min_lr=0.00001,
        verbose=1
    )

    checkpoint = ModelCheckpoint(
        'best_gru_model.keras',
        monitor='val_loss',
        save_best_only=True,
        verbose=1
    )

    # Model summary
    print("\nModel Architecture:")
    print("-" * 40)
    model.summary()

    # Train model
    print("\n" + "="*60)
    print("TRAINING MODEL")
    print("="*60)

    history = model.fit(
        x_train, y_train,
        validation_data=(x_val, y_val),
        epochs=num_epochs,
        batch_size=batch_size,
        callbacks=[early_stop, reduce_lr, checkpoint],
        verbose=1
    )

    return model, history

# Train the model
print("\n" + "="*60)
print("STARTING TRAINING")
print("="*60)

gru_model, gru_history = fixed_gru_model(
    X_train_split, y_train_split,
    X_val_split, y_val_split,
    num_epochs=50,
    batch_size=128, # Smaller batch size
    learning_rate=0.0005 # Lower learning rate
)

```

```

/* Visualization of the GRU Real and Predicted Values*/

import matplotlib.pyplot as plt

def plot_gru_results(y_true, y_pred, start_idx=1300, end_idx=1423):
    """
    Plot GRU predictions vs actual values
    """
    fig, axes = plt.subplots(3, 2, figsize=(15, 12))
    axes = axes.flatten()

    feature_names = ['Significant Wave Height', 'Maximum Wave Height',
                     'Zero Upcrossing Period', 'Peak Energy Period',
                     'Direction ( )', 'Sea Surface Temperature']
    units = ['m', 'm', 's', 's', ' ', 'C ']

    for i in range(6):
        ax = axes[i]

        # Get data slice
        real_data = y_true[start_idx:end_idx, i]
        pred_data = y_pred[start_idx:end_idx, i]

        # Plot
        ax.plot(real_data, label='Actual', color='blue', alpha=0.7, linewidth=1.5)
        ax.plot(pred_data, label='Predicted', color='red', alpha=0.7, linewidth=1.5, linestyle='--')

        # Calculate metrics
        rmse = math.sqrt(mean_squared_error(real_data, pred_data))
        mae = mean_absolute_error(real_data, pred_data)

        # Set title and labels
        ax.set_title(f'{feature_names[i]}\nRMSE: {rmse:.3f} {units[i]}, MAE: {mae:.3f} {units[i]}')
        ax.set_xlabel('Time Step')
        ax.set_ylabel(f'Value ({units[i]})')
        ax.legend()
        ax.grid(True, alpha=0.3)

        # Special handling for direction
        if i == 4: # Direction
            ax.set_ylim([0, 360])

    plt.suptitle(f'GRU Predictions vs Actual (Samples {start_idx}-{end_idx})', fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()

# Plot results

```

```
plot_gru_results(test_actual_final, test_pred_final)
```

```
/* Temporal Fusion Transformer (TFT) - Multi-Target Implementation*/
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
```

```
class GatedLinearUnit(nn.Module):
```

```
    """Gated Linear Unit for controlling information flow"""
```

```
    def __init__(self, input_size, hidden_size, dropout=0.1):
```

```
        super().__init__()
```

```
        self.fc1 = nn.Linear(input_size, hidden_size)
```

```
        self.fc2 = nn.Linear(input_size, hidden_size)
```

```
        self.dropout = nn.Dropout(dropout)
```

```
    def forward(self, x):
```

```
        sig = torch.sigmoid(self.fc1(x))
```

```
        x = self.dropout(self.fc2(x))
```

```
        return sig * x
```

```
class GatedResidualNetwork(nn.Module):
```

```
    """
```

```
    Gated Residual Network (GRN) - Key TFT component
```

```
    Provides flexible nonlinear processing with skip connections
```

```
    """
```

```
    def __init__(self, input_size, hidden_size, output_size, dropout=0.1, context_size=None)
```

```
        super().__init__()
```

```
        self.input_size = input_size
```

```
        self.output_size = output_size
```

```
        self.context_size = context_size
```

```
        self.fc1 = nn.Linear(input_size, hidden_size)
```

```
        self.elu = nn.ELU()
```

```
        self.fc2 = nn.Linear(hidden_size, output_size)
```

```
        self.dropout = nn.Dropout(dropout)
```

```
        if context_size is not None:
```

```
            self.context_fc = nn.Linear(context_size, hidden_size, bias=False)
```

```
        self.gate = nn.Linear(output_size, output_size)
```

```
        self.layer_norm = nn.LayerNorm(output_size)
```

```

        if input_size != output_size:
            self.skip_layer = nn.Linear(input_size, output_size)
        else:
            self.skip_layer = None

    def forward(self, x, context=None):
        # Skip connection
        if self.skip_layer is not None:
            skip = self.skip_layer(x)
        else:
            skip = x

        # Main pathway
        hidden = self.fc1(x)
        if context is not None and self.context_size is not None:
            hidden = hidden + self.context_fc(context)
        hidden = self.elu(hidden)
        hidden = self.fc2(hidden)
        hidden = self.dropout(hidden)

        # Gating
        gating = torch.sigmoid(self.gate(hidden))
        output = gating * hidden

        # Add & Norm
        output = self.layer_norm(output + skip)
        return output

class VariableSelectionNetwork(nn.Module):
    """
    Variable Selection Network (VSN)
    Learns to select relevant input variables
    """
    def __init__(self, input_size, num_inputs, hidden_size, dropout=0.1, context_size=None):
        super().__init__()
        self.hidden_size = hidden_size
        self.num_inputs = num_inputs
        self.input_size = input_size

        # GRN for each input variable
        self.input_grns = nn.ModuleList([
            GatedResidualNetwork(input_size, hidden_size, hidden_size, dropout)
            for _ in range(num_inputs)
        ])

        # GRN for variable selection weights
        self.selection_grn = GatedResidualNetwork(
            input_size * num_inputs, hidden_size, num_inputs, dropout, context_size
        )

```

```

def forward(self, inputs, context=None):
    # inputs: (batch, num_inputs, input_size)
    batch_size = inputs.size(0)

    # Process each variable
    processed = []
    for i, grn in enumerate(self.input_grns):
        processed.append(grn(inputs[:, i, :]))
    processed = torch.stack(processed, dim=1) # (batch, num_inputs, hidden_size)

    # Calculate selection weights
    flattened = inputs.reshape(batch_size, -1)
    weights = self.selection_grn(flattened, context)
    weights = F.softmax(weights, dim=-1).unsqueeze(-1) # (batch, num_inputs, 1)

    # Weighted sum
    output = (processed * weights).sum(dim=1) # (batch, hidden_size)
    return output, weights

class InterpretableMultiHeadAttention(nn.Module):
    """
    Interpretable Multi-Head Attention
    Modified attention that shares values across heads for interpretability
    """
    def __init__(self, hidden_size, num_heads, dropout=0.1):
        super().__init__()
        self.hidden_size = hidden_size
        self.num_heads = num_heads
        self.head_size = hidden_size // num_heads

        self.q_linear = nn.Linear(hidden_size, hidden_size)
        self.k_linear = nn.Linear(hidden_size, hidden_size)
        self.v_linear = nn.Linear(hidden_size, hidden_size)
        self.out_linear = nn.Linear(hidden_size, hidden_size)

        self.dropout = nn.Dropout(dropout)
        self.scale = self.head_size ** -0.5

    def forward(self, query, key, value, mask=None):
        batch_size, seq_len, _ = query.size()

        # Linear projections
        Q = self.q_linear(query).view(batch_size, seq_len, self.num_heads, self.head_size).t
        K = self.k_linear(key).view(batch_size, seq_len, self.num_heads, self.head_size).t
        V = self.v_linear(value).view(batch_size, seq_len, self.num_heads, self.head_size).t

        # Attention scores
        scores = torch.matmul(Q, K.transpose(-2, -1)) * self.scale

```

```

        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)

        attn_weights = F.softmax(scores, dim=-1)
        attn_weights = self.dropout(attn_weights)

        # Apply attention to values
        context = torch.matmul(attn_weights, V)
        context = context.transpose(1, 2).contiguous().view(batch_size, seq_len, self.hidden)

        output = self.out_linear(context)
        return output, attn_weights

print("TFT Core Components Defined:")
print("    Gated Linear Unit (GLU)")
print("    Gated Residual Network (GRN)")
print("    Variable Selection Network (VSN)")
print("    Interpretable Multi-Head Attention")

/* TFT: Visualization of Real and Predicted Values*/

plt.figure(figsize=(17, 25))

plt.subplot(6, 2, 1)
plt.plot(tft_prediction[1300:, 0], color='red', alpha=0.7, label='prediction')
plt.plot(y_test_tft_original[1300:, 0], color='blue', alpha=0.5, label='real')
plt.title('Significant Wave Height')
plt.legend()
plt.grid(True, axis='y')

plt.subplot(6, 2, 2)
plt.plot(tft_prediction[1300:, 1], color='red', alpha=0.7, label='prediction')
plt.plot(y_test_tft_original[1300:, 1], color='blue', alpha=0.5, label='real')
plt.title('Maximum Wave Height')
plt.legend()
plt.grid(True, axis='y')

plt.subplot(6, 2, 3)
plt.plot(tft_prediction[1300:, 2], color='red', alpha=0.7, label='prediction')
plt.plot(y_test_tft_original[1300:, 2], color='blue', alpha=0.5, label='real')
plt.title('Zero Upcrossing Wave Period')
plt.legend()
plt.grid(True, axis='y')

```

```

plt.subplot(6, 2, 4)
plt.plot(tft_prediction[1300:, 3], color='red', alpha=0.7, label='prediction')
plt.plot(y_test_tft_original[1300:, 3], color='blue', alpha=0.5, label='real')
plt.title('Peak Energy Wave Period')
plt.legend()
plt.grid(True, axis='y')

plt.subplot(6, 2, 5)
plt.plot(tft_prediction[1300:, 4], color='red', alpha=0.7, label='prediction')
plt.plot(y_test_tft_original[1300:, 4], color='blue', alpha=0.5, label='real')
plt.title('Direction Related to True North')
plt.legend()
plt.grid(True, axis='y')

plt.subplot(6, 2, 6)
plt.plot(tft_prediction[1300:, 5], color='red', alpha=0.7, label='prediction')
plt.plot(y_test_tft_original[1300:, 5], color='blue', alpha=0.5, label='real')
plt.title('Sea Surface Temperature')
plt.legend()
plt.grid(True, axis='y')

plt.suptitle('TFT: Real vs Predicted Values (Samples 1300+)', fontsize=14, y=1.01)
plt.tight_layout()
plt.show()

/* Model Comparison: LSTM vs GRU vs TFT**/

import math
import numpy as np
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

def calculate_metrics(y_true, y_pred):
    """Calculate RMSE, MAE, MAPE, and R for each feature"""
    feature_names = ['Hs', 'Hmax', 'Tz', 'Tp', 'Direction', 'SST']
    metrics = {}

    for i, feat in enumerate(feature_names):
        rmse = math.sqrt(mean_squared_error(y_true[:, i], y_pred[:, i]))
        mae = mean_absolute_error(y_true[:, i], y_pred[:, i])
        # Avoid division by zero in MAPE
        mask = y_true[:, i] != 0
        mape = np.mean(np.abs((y_true[:, i][mask] - y_pred[:, i][mask]) / y_true[:, i][mask]))
        r2 = r2_score(y_true[:, i], y_pred[:, i])
        metrics[feat] = {'RMSE': rmse, 'MAE': mae, 'MAPE': mape, 'R2': r2}

```



```

# Overall metrics
overall_rmse = math.sqrt(mean_squared_error(y_true, y_pred))
overall_mae = mean_absolute_error(y_true, y_pred)
mask_all = y_true.flatten() != 0
overall_mape = np.mean(np.abs((y_true.flatten()[mask_all] - y_pred.flatten()[mask_all]))
overall_r2 = r2_score(y_true.flatten(), y_pred.flatten())
metrics['Overall'] = {'RMSE': overall_rmse, 'MAE': overall_mae, 'MAPE': overall_mape, 'R

return metrics

# Calculate metrics for each model
# LSTM: test_prediction, y_test
# GRU: test_pred_final, test_actual_final
# TFT: tft_prediction, y_test_tft_original

lstm_metrics = calculate_metrics(y_test, test_prediction)
gru_metrics = calculate_metrics(test_actual_final, test_pred_final)
tft_metrics = calculate_metrics(y_test_tft_original, tft_prediction)

# COMPARISON TABLE

print("="*90)
print("MODEL COMPARISON: RMSE (Root Mean Squared Error)")
print("="*90)
print(f"{'Feature':<15} {'LSTM':<15} {'GRU':<15} {'TFT':<15} {'Best Model':<15}")
print("-"*90)

feature_names = ['Hs', 'Hmax', 'Tz', 'Tp', 'Direction', 'SST', 'Overall']
for feat in feature_names:
    lstm_val = lstm_metrics[feat]['RMSE']
    gru_val = gru_metrics[feat]['RMSE']
    tft_val = tft_metrics[feat]['RMSE']

# Find best (lowest RMSE)
best = min(lstm_val, gru_val, tft_val)
if best == lstm_val:
    best_model = "LSTM"
elif best == gru_val:
    best_model = "GRU"
else:
    best_model = "TFT"

if feat == 'Overall':
    print("-"*90)
    print(f"{{feat:<15}} {{lstm_val:<15.4f}} {{gru_val:<15.4f}} {{tft_val:<15.4f}} {{best_model:<15}}")

print("="*90)

```

```
# MAE COMPARISON
```

```
print("\n" + "="*90)
print("MODEL COMPARISON: MAE (Mean Absolute Error)")
print("="*90)
print(f"{'Feature':<15} {'LSTM':<15} {'GRU':<15} {'TFT':<15} {'Best Model':<15}")
print("-"*90)

for feat in feature_names:
    lstm_val = lstm_metrics[feat]['MAE']
    gru_val = gru_metrics[feat]['MAE']
    tft_val = tft_metrics[feat]['MAE']

    best = min(lstm_val, gru_val, tft_val)
    if best == lstm_val:
        best_model = "LSTM    "
    elif best == gru_val:
        best_model = "GRU      "
    else:
        best_model = "TFT      "

    if feat == 'Overall':
        print("-"*90)
        print(f"{'feat':<15} {'lstm_val':<15.4f} {'gru_val':<15.4f} {'tft_val':<15.4f} {'best_model':<15}")

print("="*90)
```

```
# MAPE COMPARISON
```

```
print("\n" + "="*90)
print("MODEL COMPARISON: MAPE (Mean Absolute Percentage Error)      Lower is Better (%)")
print("="*90)
print(f"{'Feature':<15} {'LSTM':<15} {'GRU':<15} {'TFT':<15} {'Best Model':<15}")
print("-"*90)

for feat in feature_names:
    lstm_val = lstm_metrics[feat]['MAPE']
    gru_val = gru_metrics[feat]['MAPE']
    tft_val = tft_metrics[feat]['MAPE']

    best = min(lstm_val, gru_val, tft_val)
    if best == lstm_val:
        best_model = "LSTM    "
    elif best == gru_val:
        best_model = "GRU      "
    else:
```

```

        best_model = "TFT"

    if feat == 'Overall':
        print("-"*90)
        print(f"{feat:<15} {lstm_val:<15.4f} {gru_val:<15.4f} {tft_val:<15.4f} {best_model:<15}")

print("-"*90)

# R COMPARISON

print("\n" + "-"*90)
print("MODEL COMPARISON: R Score (Higher is Better)")
print("-"*90)
print(f"{'Feature':<15} {'LSTM':<15} {'GRU':<15} {'TFT':<15} {'Best Model':<15}")
print("-"*90)

for feat in feature_names:
    lstm_val = lstm_metrics[feat]['R2']
    gru_val = gru_metrics[feat]['R2']
    tft_val = tft_metrics[feat]['R2']

    # For R , higher is better
    best = max(lstm_val, gru_val, tft_val)
    if best == lstm_val:
        best_model = "LSTM"
    elif best == gru_val:
        best_model = "GRU"
    else:
        best_model = "TFT"

    if feat == 'Overall':
        print("-"*90)
        print(f"{feat:<15} {lstm_val:<15.4f} {gru_val:<15.4f} {tft_val:<15.4f} {best_model:<15}")

print("-"*90)

# SUMMARY

print("\n" + "-"*90)
print("FINAL SUMMARY")
print("-"*90)

# Count wins for each model
lstm_wins = sum(1 for feat in feature_names[:-1] if lstm_metrics[feat]['RMSE'] == min(lstm_m
gru_wins = sum(1 for feat in feature_names[:-1] if gru_metrics[feat]['RMSE'] == min(lstm_met
tft_wins = sum(1 for feat in feature_names[:-1] if tft_metrics[feat]['RMSE'] == min(lstm_met

```

```

print(f"Features won (by RMSE): LSTM={lstm_wins}, GRU={gru_wins}, TFT={tft_wins}")
print(f"\nOverall RMSE: LSTM={lstm_metrics['Overall']['RMSE']:.4f}, GRU={gru_metrics['Overa
print(f"Overall MAE: LSTM={lstm_metrics['Overall']['MAE']:.4f}, GRU={gru_metrics['Overall'
print(f"Overall MAPE: LSTM={lstm_metrics['Overall']['MAPE']:.4f}%, GRU={gru_metrics['Overal
print(f"Overall R : LSTM={lstm_metrics['Overall']['R2']:.4f}, GRU={gru_metrics['Overall'

# Determine overall best model
overall_rmse_vals = {'LSTM': lstm_metrics['Overall']['RMSE'],
                    'GRU': gru_metrics['Overall']['RMSE'],
                    'TFT': tft_metrics['Overall']['RMSE']}
best_overall = min(overall_rmse_vals, key=overall_rmse_vals.get)
print(f"\n Best Overall Model (by RMSE): {best_overall}")
print("="*90)

```

Appendix B- Conference details



Acceptance Letter

Paper Title: Multi-Target Ocean Wave Forecasting Using Deep Learning: Comparative Analysis of Recurrent and Transformer-Based Architectures

Acceptance id: ICICI-035

Authors: Nviz Yadav, Aritra Nanda, Saptak Singha, Mehfooza M

Dear Authors,

On behalf of the Conference committee, we would like to congratulate you on your article to the **ICICI 2026 IEEE Conference**, which will be held from **10-12, June 2026** at **S.E.A College of Engineering and Technology**, Bengaluru, Karnataka, India. You have been selected to deliver your presentation at the 4th International Conference on Inventive Computing and Informatics - ICICI 2026.

As a result of the review and results, we are pleased inform that you can now submit the camera-ready paper for inclusion into the ICICI proceedings. We appreciate if you could send the final version of your research paper at your earliest convenience, in order to ensure the timely publication. When submitting your final paper, please highlight the changes made according to the review comments.

Best Regards,

A circular blue stamp with 'ICICI' in the center and '2026' around it.

Dr. Usha Desai
Organizing Secretary
ICICI 2026

Proceedings by



Figure 8.1: Conference Acceptance Letter